Computationally and Communication Efficient Batched Asynchronous DPSS from Lightweight Cryptography

Akhil Bandarupalli*, Xiaoyu Ji†, Soham Jog*, Aniket Kate*‡, Chen-Da Liu-Zhang§¶, Yifan Song†‖
*Purdue University {abandaru,sjog,aniket}@purdue.edu
†Tsinghua University jixy23@mails.tsinghua.edu.cn, yfsong@mail.tsinghua.edu.cn
§ Lucerne University of Applied Sciences and Arts chen-da.liuzhang@hslu.ch
‡ Supra Research,¶Web3 Foundation, ‖Shanghai Qi-Zhi Institute

***Abstract*—Verifiable Secret Sharing (VSS) is a fundamental primitive in threshold cryptography and multi-party computation. It preserves secrecy, integrity, and availability of a shared secret for a fixed set of parties, with a subset of them being malicious. In practical applications, especially when the secret sharing is expected to be maintained over long durations, the VSS scheme should be able to cater to a dynamic setting where involved parties may change. The primitive known as *Dynamic Proactive Secret Sharing* (DPSS) is beneficial here, as it facilitates the secure transfer of secrets from the original committee to a new committee. Nevertheless, prior works on DPSS protocols either rely on unrealistic time bounds on message delivery or have a high computational cost, which limits their scalability beyond tens of parties.**

In this work, we present a suite of scalable Asynchronous Dynamic Proactive Secret Sharing (DPSS) protocols that utilize lightweight cryptographic tools, such as hash functions. The protocols achieve full security and optimal fault tolerance with amortized linear communication costs. Unlike existing solutions, our proposed protocols are also post-quantum secure. By balancing computation and communication, our approach offers practical performance at scale. Our implementation results demonstrate improved scalability and efficiency, surpassing the current state-of-the-art achieving a $22.1\times$ lower latency than the prior best work. Furthermore, our solution also scales gracefully with increasing n and reshares a batch of $100,000$ secrets between committees of sizes $n = 112$ parties in under a minute.

1. Introduction

Verifiable Secret Sharing (VSS) [15], [43] is a foundational primitive in distributed cryptography. In recent years, VSS has seen renewed interest in the context of decentralization, particularly from blockchain platforms and web3 systems [9], [21], [25], [28], [36], [42], [44]. These systems leverage VSS to securely distribute and store secrets among participating parties, ensuring secrecy, availability, and integrity even in the presence of a malicious minority. As a result, VSS has become a core building block for a wide range of applications in this space, including secure multi-party computation (MPC) protocols [38], [50], secret-shared databases [9], [51], and threshold cryptographic schemes [23], [36], [44].

A key challenge faced by modern decentralized systems, however, is preserving secrets as the set of participating parties evolves over time. For instance, Blockchains with mobile participation, such as Ethereum [52] and Algorand [29], frequently experience nodes joining and leaving the network. Preserving the core properties of VSS in these environments requires robustness to participant churn, varying numbers of corrupt parties, and adaptive fault thresholds. At the same time, solutions in this space must scale to meet growing real-world demands, both in terms of supporting hundreds of validators typical of blockchain platforms, and in managing potentially hundreds of thousands of secrets as such technologies are increasingly adopted [51].

Dynamic Committee Proactive Secret Sharing (DPSS) [11], [30], [46], [51], [55] is a cryptographic primitive designed for systems with dynamic participation. DPSS facilitates a set of parties maintaining secrets, called the old or outgoing committee, to transfer them to an incoming or new committee, while preserving the secrecy, integrity, and availability of secrets in the presence of a minority of faulty parties in both committees. In this work, we consider DPSS in an asynchronous network to ensure security and performance in real-world networks like the Internet. Asynchronous DPSS protocols can transfer secrets while tolerating up to one-third of the faulty parties in each committee.

However, prior protocols in this setting [11], [46], [51], [55] suffer from a substantial scalability problem. This problem arises mainly because of their high computation overhead from additively homomorphic, public-key cryptographic operations such as Discrete-Log exponentiations. For instance, the prior best works Cobra [51] [IEEE S&P '22] and Long Live [55] [USENIX Security'23] require $O(n^2)$ Discrete-Log exponentiations per secret transfer where n is the number of parties in the incoming committee. Given that each exponentiation takes around 50 microseconds, in a setting with $n = 100$ parties, the computation time per secret is around 0.5 seconds in a single-core machine. Therefore, this computation overhead becomes untenable for a decent batch size of $L = 100000$ secrets. In fact, Cobra reported that computation from the use of public-

Table 1: Comparison of relevant DPSS protocols.

Protocol	Network	Post Quantum Security	Cryptographic Computation Complexity (per party)	Communication Complexity (per party): Amortized (per secret)	Communication Complexity (per party): Additive Overhead	Cost to Completion	Round Complexity	Storage (per party)	Setup	Cryptographic Assumption
Cobra [51]	p sync	✗	$\mathcal{O}(ntL)$ DLE	$\mathcal{O}(t)$	$\mathcal{O}(\kappa n^3)$	$\mathcal{O}(t)$	$\mathcal{O}(t)$	$\mathcal{O}(\kappa nL)$	PKI	DLog
MPSS [46]	p sync	✗	$\mathcal{O}(n^2tL)$ DLE	$\mathcal{O}(nt)$	$\mathcal{O}(\kappa n^3)$	$\mathcal{O}(nt)$	$\mathcal{O}(t)$	$\mathcal{O}(\kappa n^2 L)$	PKI	DLog
Cachin *et al* [11]	async	✗	$\mathcal{O}(n^3L)$ DLE	$\mathcal{O}(n^2)$	$\mathcal{O}(\kappa n^3)$	—	$\mathcal{O}(1)$	$\mathcal{O}(\kappa nL)$	Secure channels	DLog
Long Live [55]	async	✗	$\mathcal{O}(ntL)$ DLE†	$\mathcal{O}(t)$	$\mathcal{O}(\kappa n^2)$	$\mathcal{O}(t)$	$\mathcal{O}(1)$	$\mathcal{O}(\kappa nL)$	SRS + PKI	SXDH
Ours	async	✓	$\mathcal{O}(nL)$ LWC	$\mathcal{O}(t)$	$\mathcal{O}(\kappa n^2 \log^2(n))$	$\mathcal{O}(t)$	$\mathcal{O}(1)$	$\mathcal{O}(\kappa L)$ +$\mathcal{O}(\kappa n^2 \log^2(n))$	Secure channels	RO
Ours	async	✓	$\mathcal{O}(nL)$ LWC	$\mathcal{O}(1)$	$\mathcal{O}(\kappa n^2 \log^2(n))$	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(\kappa L)$ +$\mathcal{O}(\kappa n^2 \log^2(n))$	Secure channels	RO

In the table, n is the total number of parties, $t < \frac{n}{3}$ is the actual number of faulty parties, L is the total number of secrets being reshared, and κ is the cryptographic security parameter. **Cryptography cost**: The overall computational cost of prior DPSS protocols is dominated by cryptographic operations. For instance, each discrete-log exponentiation (DLE) is $\approx 1000\times$ more expensive than a Lightweight cryptographic (LWC) operation like a hash computation. Therefore, our protocol is concretely computationally efficient. Furthermore, the cryptography cost incurred by our protocol is independent of the number of secrets L. **Communication**: The amortized cost approximates the overall communication per secret (in number of field elements) when L is large. The additive overhead measures the communication incurred independent of number of secrets L. The overall communication for our protocol, for instance, is $O(nL + \kappa n^3 \log^2(n))$ bits. The cost to completion is the cost to recover shares of the secret to each party. **Optimistic Execution**: Denotes executions where the number of faulty parties acting maliciously is small or $t \sim \mathcal{O}(1)$. **Storage Cost**: We compare the per-party storage cost incurred in bits. Note that each party must store at least one share per secret, which adds an additional $\mathcal{O}(L)$ factor to this cost. **Setup**: Our protocols require secure channels among parties. † The Structured Random String (SRS) setup has been classified as a trusted setup in prior work [55] because it needs a Powers-of-Tau ceremony. Without this setup, the computation and communication costs for Long Live [55] increase to quadratic per secret.

key cryptography itself takes up around 87% of the overall runtime of their protocol, which is over 10 minutes for $L = 100000$ secrets with $n = 10$ parties.

In this work, we ask the following question: *Is it possible to build a new asynchronous DPSS protocol that overcomes these scalability bottlenecks and can reshare hundreds of thousands of secrets in systems with $n > 100$ participants?*

1.1. Our Results

We answer this question affirmatively and present a novel asynchronous DPSS protocol and address the scalability problem by focusing on computational efficiency. We design our protocol using only computationally efficient or lightweight cryptography [7], [8], [39], [49]. Our protocol only employs tools like Hash computations and authenticated symmetric-key encryption, which are at least two orders of magnitude faster than Discrete-Log exponentiations, and are therefore lightweight in nature. In the following results overview, we assume each secret to be from a finite field and measure communication costs in the number of field elements. For reference, we instantiate our protocol with a 252-bit finite field. We compare the complexities of our protocol with prior works in Table 1.

Succinctly explaining the improvements, our protocol is computationally more efficient than prior works, mainly from the lower computation cost of lightweight cryptographic tools. On top of achieving lesser cryptographic overhead per secret, our protocol also requires only a linear or $\mathcal{O}(n)$ field elements of amortized communication per secret. Both these improvements lead to minimal resource use per secret, which enables our protocol to share substantially more secrets at all values of n.

On the other hand, our protocol has a communication additive overhead independent of the number of secrets of $\mathcal{O}(\kappa n^3 \log^2(n))$ bits, a $\log^2(n)$ factor higher than prior work relying on Discrete-Log cryptography. Additionally, our protocol also has a higher round complexity of $\mathcal{O}(n)$ rounds, compared to $\mathcal{O}(1)$ rounds for the prior best work.

Through extensive experimental evaluation, we demonstrate that these tradeoffs overwhelmingly skew our work towards improved practical performance, achieving a $23\times$ lower latency than the prior best works in this space [51], [55]. We discuss some key performance metrics below.

Computational Efficiency. In terms of improvements over prior works, our protocol requires $\mathcal{O}(n^2 \log^2(n))$ lightweight cryptographic operations for resharing a batch of L secrets when $t = \frac{n-1}{3}$ parties behave maliciously. Although the number of LWC operations are independent of L, the overall computation complexity is still $O(nL)$. This is mainly because the computation complexity of LWC instantiations in practice such as SHA256 Hash function still

increases linearly with input size. We describe this cost in detail in Section 7. In contrast, prior protocols [51], [55] require at least $\mathcal{O}(n^2)$ Discrete Log exponentiations per secret, indicating at least a 10^4 factor concrete improvement in computation cost for batches of size $L > \mathcal{O}(\log^2(n))$.

Amortized linear communication. Additionally, our protocol requires only $\mathcal{O}(n)$ field elements of communication per secret resharing or $\mathcal{O}(1)$ field elements per party per secret. In contrast, prior works [51], [55] require at least $\mathcal{O}(n^2)$ communication per secret or $\mathcal{O}(n)$ field elements per party when $t = \frac{n-1}{3}$ behave in a malicious manner, indicating a linear factor improvement over these works. This low asymptotic cost per secret makes our protocol favorable for applications with large numbers of secrets like Multi-Party Computation with large circuit sizes and large secret-shared databases. However, our protocol's additive communication overhead term is $\mathcal{O}(\log^2(n))$ factor higher than the prior best work, Long Live.

Post-Quantum Security. In the impending era of quantum computers, protocols must be secure against potential adversaries with access to quantum computers. Our protocol is Post-Quantum Secure because lightweight cryptographic tools like the SHA256 Hash function and AES symmetric key cipher are expected to retain their advantage even against a polynomial-time quantum adversary [31]. In contrast, prior DPSS protocols' reliance on the Discrete-Log hardness assumption renders them insecure against a quantum adversary because of Shor's algorithm [48], which can solve such problems within polynomial time.

Round Complexity. Our protocol requires $\mathcal{O}(n)$ rounds to terminate when $t = \frac{n-1}{3}$ parties behave maliciously. Nevertheless, we also provide an alternate construction to manage round complexity and trade rounds for asymptotic communication per secret. This protocol terminates in constant rounds, but pays quadratic communication per secret, matching the asymptotic complexities of Long Live. We explore this tradeoff in detail in our experiment section.

Storage. On top of computation, communication, and round complexities, parties in DPSS must also store details about secret shares and protocol transcripts. This storage cost is important in multiple applications, especially in blockchains where parties maintain these secrets over long periods. Our protocol requires parties in the incoming committee to store the protocol's transcripts, including commitments and symmetric key ciphertexts, and therefore has storage overhead of $\mathcal{O}(L+\kappa n^2 \log^2(n))$ bits per party. In comparison, protocols like Cobra and Long Live must store at least $\mathcal{O}(\kappa n L)$ bits for L secrets. Essentially, for $L > n \log^2(n)$ secrets, our storage cost is lower than these prior works.

Optimistic case complexities. We define an optimistic execution as having only $\mathcal{O}(1)$ parties behaving maliciously. Here, we note that we do not consider crash faults - parties that crashed or are unresponsive - as malicious. Malicious behavior implies active Byzantine behavior sending invalid shares from higher degree polynomials, broadcasting invalid commitments, and so on. In such cases, the computation complexity of our protocol further improves by a $\mathcal{O}(n)$ factor, thus resulting in fast execution. Both Cobra and Long Live also see similar improvements in their performance under such scenarios.

Evaluation We provide an end-to-end implementation of our protocols in Rust. Through extensive experimental evaluation over a geo-distributed testbed, we demonstrate at least $22.1\times$ lower latency of our pessimistic case execution (with $t = \frac{n-1}{3}$ faulty parties) than the optimistic cases of prior works Long Live and Cobra [51], [55]. Further, our implementation also makes DPSS practical, resharing hundreds of thousands of secrets with $n = 112$ parties under a minute, and is the first DPSS protocol to demonstrate practicality at this scale.

2. Technical Overview

2.1. System model

DPSS is used to transfer an already-shared secret s from an old committee $\mathsf{P}_{\mathtt{old}}$ of n_1 parties to a new committee $\mathsf{P}_{\mathtt{new}}$ of n_2 parties. Every pair of parties can interact over secure (confidential and authenticated) point-to-point channels. We assume the asynchronous communication setting where messages by honest parties may be delayed by an arbitrary but unknown finite time. Moreover, messages may arrive in an arbitrary order. The only guarantee is that all the messages sent by an honest party will eventually be delivered.

A transfer between the old and new committees happens at the well-defined epoch boundary. We follow the same epoch definition as Long Live [55]. The system is defined by a period called an epoch, where in each epoch e, a fixed set of parties P_e is responsible for maintaining the secret. In the asynchronous communication setting, different parties may enter and leave different epochs at different physical times. The epoch e is considered active from the moment the first honest party in either committee enters it to the moment the last honest parties in both committees exit it. For cases where only few parties of an epoch change, any party in an epoch e triggering a share transfer procedure and also is part of the incoming epoch $e+1$ enters a transitional period between epochs e and $e+1$, where it is considered to be part of both epochs. Such parties delete all its shares from epoch e only after terminating the resharing procedure and exits epoch e to move to epoch $e+1$.

Adversary Model. We assume a probabilistic polynomial-time (PPT) adversary A that controls parties and can make them behave arbitrarily maliciously. A controls up to $t_1 < \frac{n_1}{3}$ parties in the outgoing committee and up to $t_2 < \frac{n_2}{3}$ parties in the incoming committee. In case a corrupted party belongs to both outgoing and incoming committees, it contributes to the corruption thresholds in both committees. A can delay the delivery of messages between honest parties by any finite amount of time, however, it cannot modify or drop those messages.

Problem Definition. For security, we use the UC framework [12] to define the security of our protocols.[1] We adopt the definition of DPSS from Long Live [55]. In line with the UC framework, we define the ideal functionality $\mathcal{F}_{\mathsf{DPSS}}$ in Section 2.1. Essentially, the functionality ensures the following properties.

- **Correctness**: For each secret $s_1, \ldots, s_L$ held by the outgoing committee, every honest party P_i in $\mathsf{P}_{\mathtt{new}}$ must eventually output shares $f_j(\alpha_i)$ of degree-t_2 polynomials $f_j(x)$ such that $f_j(\alpha_0) = s_j$ for $j \in \{1, \ldots, L\}$.
- **Secrecy**: For any PPT adversary A controlling t_1, t_2 parties in $\mathsf{P}_{\mathtt{old}}, \mathsf{P}_{\mathtt{new}}$, there exists a PPT simulator $\mathcal{S}$ such that the output of $\mathcal{S}$ and A must be indistinguishable.
- **Termination**: If the adversary A controls at most t_1, t_2 parties in $\mathsf{P}_{\mathtt{old}}, \mathsf{P}_{\mathtt{new}}$, then all honest parties in both committees must eventually terminate the protocol.

Functionality $\mathcal{F}_{\mathsf{DPSS}}$

Public Input: N

$\mathcal{F}_{\mathsf{DPSS}}$ runs with party sets $\mathsf{P}_{\mathtt{old}}, \mathsf{P}_{\mathtt{new}}$, and an adversary $\mathcal{S}$. Parties in $\mathsf{P}_{\mathtt{old}}$ possess shares of N secrets $\{s_1, \ldots, s_L\}$. Denote the corruption threshold in $\mathsf{P}_{\mathtt{old}}, \mathsf{P}_{\mathtt{new}}$ as t_1, t_2.

1: Upon receiving $t_1 + 1$ shares of $[s_1]_t, \ldots, [s_L]_t$ from honest parties in $\mathsf{P}_{\mathtt{old}}$, it computes the secrets $s_1, \ldots, s_L$.

2: Upon receiving request $(\mathtt{Transfer}, \mathsf{P}_{\mathtt{new}})$ from $t_1 + 1$ parties in $\mathsf{P}_{\mathtt{old}}$, randomly sample degree-t_2 polynomials $f_1(x), \ldots, f_L(x)$ such that $f_\ell(\alpha_0) = s_\ell$ for all $\ell \in [L]$. Then, send request-based delayed outputs $\mathtt{Term}$ to each party in $\mathsf{P}_{\mathtt{old}}$ and $f_1(\alpha_i), \ldots, f_L(\alpha_i)$ to each party $P_i \in \mathsf{P}_{\mathtt{new}}$.

- Upon receiving a request $(\mathtt{success}, P_i)$ and $P_i \in \mathsf{P}_{\mathtt{new}}$ from $\mathcal{S}$, if the output of P_i has not been delivered, change his output by symbol $\mathtt{success}$.

3: Set $\mathsf{P}_{\mathtt{old}} = \mathsf{P}_{\mathtt{new}}$.

2.2. Overview of Prior Works

Prior works established a three-step template for achieving DPSS [46], [51], [55].

1) **Generate Random Sharings** All parties in $\mathsf{P}_{\mathtt{old}}$ prepare random degree-t_1 and t_2 Shamir secret sharings $[r_i]_{t_1}, [r'_i]_{t_2}$ for parties in $\mathsf{P}_{\mathtt{old}}, \mathsf{P}_{\mathtt{new}}$ such that $r_i = r'_i$ for $i \in \{1, \ldots, L\}$.
2) **Blinding and Reconstructing Blinded Secrets** Parties in $\mathsf{P}_{\mathtt{old}}$ first compute their shares of $[b_i]_{t_1} = [r_i]_{t_1} + [s_i]_{t_1}$, and reconstruct secrets $\{b_1, \ldots, b_L\}$.
3) **Share Transfer** Parties in $\mathsf{P}_{\mathtt{old}}$ use Reed-Solomon Erasure codes to transfer b_i for $i \in$ to $\mathsf{P}_{\mathtt{new}}$. Then, parties in $\mathsf{P}_{\mathtt{new}}$ compute their shares of $[s_i]_{t_2} = b - [r]_{t_2}$ for $\{s_1, \ldots, s_L\}$ and terminate.

1. The standard UC framework does not model eventual delivery. We consider the UC with the eventual delivery framework variant used in prior works [17], [18], [19].

For the first step, prior works like Long Live [55] and Das *et al* [23] utilized an *Asynchronous Complete Secret Sharing* (ACSS) protocol to let each party distribute shares of a random value r in both committees. An ACSS protocol enables a party or dealer to distribute a degree-t Shamir sharing of a secret to all other parties. Mainly, ACSS provides the *completeness* property - if one honest party terminates the protocol with its share, then all honest parties will eventually terminate the protocol with their correct shares. In addition, parties also prove that they shared same secret r to both $\mathsf{P}_{\mathtt{old}}$ and $\mathsf{P}_{\mathtt{new}}$ using a *secret equivalence* protocol.

Then, parties in $\mathsf{P}_{\mathtt{old}}$ use an Asynchronous Common Subset (ACS) protocol to agree on a set of $n_1 - t_1$ successful dealers who distributed sharings to both committees. Finally, they extract $t_1 + 1$ random sharings from this set using the hyperinvertible Vandermonde matrix technique in [20]. In essence, let $\boldsymbol{M}$ be a hyperinvertible Vandermonde matrix of size $(t+1) \times (2t+1)$. Then, all parties locally compute the output sharing $([r_1]_{t_1}, \ldots, [r_{t+1}]_{t_1}) = \boldsymbol{M} \cdot \{[r^{(i)}]_{t_1}\}_{i \in \mathcal{D}}$, where $\mathcal{D}$ is the set of successful dealers output by ACS.

For the second step, parties in $\mathsf{P}_{\mathtt{old}}$ can reconstruct the blinded secrets by broadcasting their shares and using an Online Error Correction (OEC) algorithm like Berkelamp-Welch. Concretely, an OEC rectifies the bad shares provided by malicious parties. To rectify t_1 bad shares and reconstruct the secret, honest parties must submit $2t_1 + 1$ good shares to the algorithm, t_1 shares for rectifying the bad shares and $t_1 + 1$ shares for interpolating the polynomial. Hence, it is imperative for the reconstruction algorithm that each honest party receives its share, which is facilitated by the completeness property of the ACSS protocol.

Challenges in achieving linear communication cost This protocol template can achieve linear amortized communication cost per secret using a linear cost ACSS and secret reconstruction protocols. Given that we have an ACSS protocol, secret reconstruction can be done efficiently with linear cost using the secret encoding technique in [8], [16], [20]. However, solutions proposed by prior works to build a linear cost ACSS protocol are prohibitively expensive and cause a substantial scalability bottleneck.

For achieving linear communication cost, prior ACSS protocols [54], [55] employed additively homomorphic public-key cryptography such as KZG polynomial commitments [33] and verifiable encryption [54]. Mainly, these protocols utilize homomorphic transcripts to let each party check the correctness of the sharing using local computations, thus achieving communication efficiency. However, this approach requires at least $\mathcal{O}(n^2)$ public-key operations per sharing, which becomes a substantial scalability bottleneck. Alternate constructions like Ji *et al* [32] avoid this computational bottleneck and use only Information-Theoretic cryptography to build ACSS. However, this work experiences an $\mathcal{O}(n^{12})$ additive overhead in communication, which is also not practical.

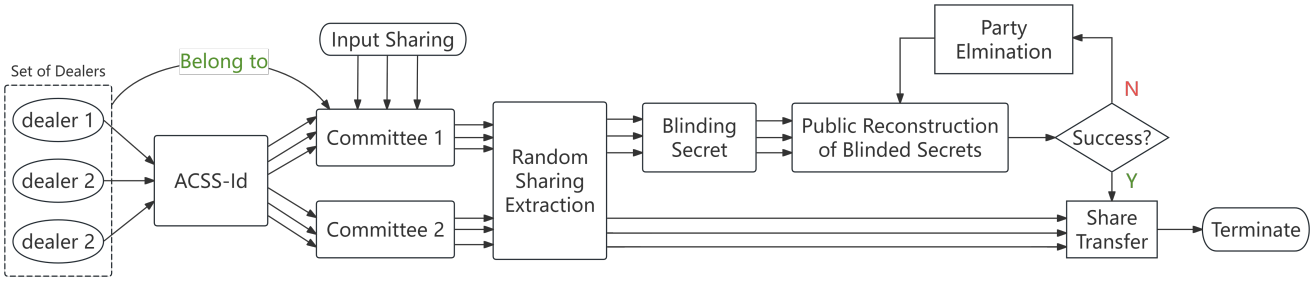


Figure 1: **Framework of Our DPSS**: The parties in the first committee will act as dealers and distribute random sharings to both committees by ACSS-Id, and all parties will extract the random sharings accordingly after they agree on a successful set of Dealers. Based on these sharings, parties in the first committee will use party elimination-based public reconstruction to reconstruct the blinded secrets, and then transfer them to parties in the second committee. With these blinded secrets, parties in the second committee can compute their output sharing and terminate.

2.3. Our Solution

We adopt the described three-step framework in our work. Mainly, our objective is to find a way around the computational cost incurred by prior ACSS protocols while achieving linear amortized communication per secret. Our starting point is the ACSS protocol in Shoup and Smart [49] based only on lightweight cryptography. This work requires $\mathcal{O}(nL+\kappa n^2\log(n))$ communication cost when the dealer is honest. However, when the dealer is malicious, this cost increases to $\mathcal{O}(n^2L+\kappa n^3\log(n))$ field elements. This occurs mainly because lightweight cryptographic tools lack transcript homomorphism, forcing honest parties to communicate more to verify and rectify bad shares issued by a malicious dealer. Therefore, employing this protocol results in quadratic cost per random sharing and DPSS.

Functionality $\mathcal{F}_{\text{ACSS-id}}$

Public Input: $(\alpha_0,\ldots,\alpha_n), t, N$
$\mathcal{F}_{\text{ACSS-id}}$ runs with parties $\mathsf{P}=\{P_1,\ldots,P_n\}$, a dealer $D\in\mathsf{P}$, and an adversary $\mathcal{S}$.

1: Upon receiving the set of corrupted parties $\mathsf{P}_{\mathcal{C}orr}$, if $D\in\mathsf{P}_{\mathcal{C}orr}$, initialize $\mathsf{P}_{\text{proof}}=\mathsf{P}_{\mathcal{C}orr}$. Otherwise, set $\mathsf{P}_{\text{proof}}=\varnothing$.
2: Upon receiving N degree-t polynomials $q_1(\cdot),\ldots,q_N(\cdot)$ from D, for each party $P_i\in\mathsf{P}$, send a request-based delayed output $q_1(\alpha_i),\ldots,q_N(\alpha_i)$ to P_i.
- Upon receiving a request $(\texttt{proof}, P_i)$ from $\mathcal{S}$, if $D\in\mathsf{P}_{\mathcal{C}orr}$ and the output of P_i has not been delivered, change the output of P_i to $(\texttt{Corrupt}, D)$ and add P_i in $\mathsf{P}_{\text{proof}}$. Otherwise, ignore this request.

3: Upon receiving request $(\texttt{Open-Proof}, P_i)$ from $t+1$ parties, do the following.
- If $P_i\in\mathsf{P}_{\text{proof}}$, send a request-based delayed output $(\texttt{Corrupt}, D)$ to all parties.
- If $P_i\in\mathsf{P}_{\mathcal{C}orr}$ and $D\notin\mathsf{P}_{\mathcal{C}orr}$, send a request-based delayed output $(\texttt{Corrupt}, P_i)$ to all parties.
- Otherwise, send all views to $\mathcal{S}$ and give up security.

4: Upon receiving `Public-Recon` from $t+1$ parties, send a requested-based delayed output $q_1(\alpha_0),\ldots,q_N(\alpha_0)$ to each $P_j\in\mathsf{P}$.

2.3.1. Review of Party Elimination Framework. We address this problem by adopting the party-elimination framework from Bandarupalli *et al* [8] to the DPSS setting. Instead of recovering the sharings distributed by the malicious parties, this framework identifies such parties and removes their contributions to the random sharings. To achieve this goal, this work proposes *ACSS with Identifiable Abort* or $\mathcal{F}_{\text{ACSS-id}}$, a new functionality described in Section 2.3. It offers a weaker completeness property where each honest party will terminate with the correct share or a verifiable proof against a malicious dealer.

Concretely, parties prepare random sharings using $\mathcal{F}_{\text{ACSS-id}}$ and attempt to reconstruct the blinded secrets. As $\mathcal{F}_{\text{ACSS-id}}$ does not guarantee share delivery to all honest parties, the naive way of reconstruction using OEC might never terminate. Hence, parties instead run an asynchronous Byzantine Agreement (BA) protocol with either their shares or accusation proofs. This protocol enables parties to either agree on the correctly reconstructed blinded secrets or a party responsible for preventing secret reconstruction.

When a malicious party P_i is detected, the framework publicly reconstructs the secrets r_i distributed by this party. Subsequently, they replace the shares $[r_i]_{t_1}$ with the reconstructed secrets r_i, recompute their shares, and eliminate the malicious party. This ensures that the reconstruction of blinded secrets will not fail again because of this party. Hence, after eliminating all faulty parties, the reconstruction phase eventually succeeds.

However, the key step in this approach is to efficiently reconstruct the secrets distributed by malicious parties with only linear communication. To achieve this task, the framework employs *Asynchronous Verifiable Secret Sharing* (AVSS), a weaker primitive than ACSS, which guarantees only $t+1$ honest parties receive their shares. Hence, as OEC cannot be used for secret reconstruction, each honest shareholder must provide a valid proof of correctness to

enable recipients to distinguish between honest and malicious shares. Note that unlike ACSS, AVSS-based sharings cannot be linearly combined without homomorphic proofs of correctness. The AVSS protocol in [8] facilitates secret sharing and reconstruction with only amortized linear cost.

Overall, each party shares random values both ways - an ACSS-Id and an AVSS protocol. During the reconstruction phase, if a malicious party is identified, parties reconstruct its shared secrets using AVSS, eliminate the malicious party, and retry the reconstruction phase, until they successfully reconstruct the blinded secrets and transfer the secrets to $\mathsf{P}_{\mathtt{new}}$.

2.3.2. Improving the Additive Overhead. Even though this framework achieves an amortized linear cost, it incurs a high communication additive overhead of $\mathcal{O}(\kappa n^4)$ bits, which hinders its practicality as n increases. This cost comes from two sources. First, the linear cost AVSS protocol in Bandarupalli *et al* [8] has an additive overhead of $\mathcal{O}(\kappa n^3)$ bits. Second, this framework utilizes an ACS protocol for achieving agreement in the reconstruction phase. The current best ACS protocol requires $\mathcal{O}(\kappa n^3)$ communication and running it for $\mathcal{O}(n)$ times to remove $\mathcal{O}(n)$ malicious parties costs $\mathcal{O}(\kappa n^4)$ bits.

Linear Cost AVSS with sub-cubic additive overhead Digging deeper into the first issue, enabling linear cost reconstruction with AVSS requires a dealer to divide its secrets into $\frac{L}{t}$ batches, each with t secrets. The idea is to form a degree-t polynomial $f_i(x)$ with secrets from each batch as coefficients and reconstruct the point $f_i(\alpha_j)$ to each party P_j. Subsequently, parties can reconstruct the whole polynomial $f_i(x)$ using OEC with only linear cost.

However, to reconstruct $f_i(\alpha_j)$, P_j must be able to distinguish between correct and incorrect shares, for which it requires proofs of correctness on each share of $f_i(\alpha_j)$. Remember that even though parties can linearly combine their shares to generate shares of $f_i(\alpha_j)$, they cannot linearly combine proofs in the lightweight cryptography setting. Hence, the dealer provides proofs of correctness for shares of $f_i(\alpha_j)$ to all parties in the sharing phase.

Generating these proofs entails cryptographic commitments on these shares and Hash-based Distributed Zero Knowledge (DZK) proofs [3], [49]. Dealer needs to commit and provide proofs for $\mathcal{O}(n^2)$ shares in each batch, for which Bandarupalli *et al* [8] used a matrix of commitments and linear-sized DZK proofs from Shoup and Smart [49]. We improve this cost by using Merkle proofs for commitments and an $\mathcal{O}(\log^2(n))$ sized DZK proofs from a recent work [13]. Overall, these optimizations improve the additive overhead from $\mathcal{O}(\kappa n^3)$ to $\mathcal{O}(\kappa n^2 \log^2(n))$ bits.

Efficient Agreement Phase In the agreement phase, parties either need to agree on the blinded secrets or on a malicious party that distributed bad shares. We improve this cost by running two protocols in parallel: (a) An asynchronous binary extension BA protocol [41], which parties use to either agree on the correct on the blinded secrets or a failure symbol $\perp$, (b) An asynchronous Multi-Valued Byzantine Agreement (MVBA) protocol [22], [35] for agreeing on a malicious party.

Concretely, parties divide the blinded secrets $\{b_1, \ldots, b_L\}$ into $\mathcal{O}(n)$ batches and conduct reconstruction in *segments*. In each segment, parties first attempt to reconstruct $\{b_1, \ldots, b_{\frac{L}{n}}\}$ using OEC. If OEC succeeds for a party P_j, it inputs $\{b_1, \ldots, b_{\frac{L}{n}}\}$ to the first protocol. If instead P_j has proofs against dealers $\mathcal{D}_j$, it uses reliable broadcast (RBC) to broadcast $\mathcal{D}_j$ and inputs $\perp$ to the first protocol. If this part succeeds, then parties terminate the segment with $\{b_1, \ldots, b_{\frac{L}{n}}\}$. Overall, the first protocol's cost in each segment is $\mathcal{O}(n \cdot \frac{L}{n}) = \mathcal{O}(L)$ field elements plus $\mathcal{O}(\kappa n^2 \log(n))$ bits.

If the first protocol output $\perp$, then each party waits until it receives a set of accused dealers $\mathcal{D}_j$ from party P_j through RBC. Then, each party inputs a disputed pair $(P_k, P_j), P_k \in \mathcal{D}_j$ to the second protocol. The MVBA protocol requires inputs to be validated by a validity oracle. In this case, parties use RBC of P_j to check if (P_k, P_j) is a correct dispute pair. Then, on getting (P_k, P_j) as output, parties reconstruct P_j's shares (of size $\mathcal{O}(\frac{L}{n})$) issued by P_k to all parties using AVID and verify it using the DZK proofs distributed by P_k. If the shares are incorrect, then parties regard P_k as malicious, and if P_j wrongly blamed P_k, then P_j is malicious. Upon identifying a malicious party, parties trigger party elimination framework and rerun the segment. Overall, the protocol terminates within $\mathcal{O}(n)$ segment runs.

Explaining our choice of MVBA, for the first $O(1)$ segments where first output was $\perp$, we use OciorMVBA [14], which requires $\mathcal{O}(\kappa n^2 \log(n))$ bits and $\mathcal{O}(\log(n))$ rounds. After eliminating $\mathcal{O}(1)$ malicious parties, we switch to a more efficient Reducer++ in Komatovic *et al* [35], which achieves the same cost in $\mathcal{O}(1)$ rounds. This protocol requires $t < (\frac{1}{3} - \epsilon)n$ resilience, so eliminating $\mathcal{O}(1)$ malicious parties allows us to use it for subsequent segments. Overall, this protocol requires $\mathcal{O}(n \cdot n \cdot \frac{L}{n}) = \mathcal{O}(nL)$ field elements plus $\mathcal{O}(\kappa n^3 \log(n))$ bits of communication, and $\mathcal{O}(\log(n)).\mathcal{O}(1) + \mathcal{O}(n) = \mathcal{O}(n)$ rounds.

2.3.3. Secret Equivalence Protocol. An issue we omitted in the overview is the problem of issuing the same secret across both committees. Remember that if malicious parties share different values through ACSS-Id across $\mathsf{P}_{\mathtt{old}}, \mathsf{P}_{\mathtt{new}}$, the protocol's framework will not work. The authors in previous works [11], [51], [55] use homomorphic commitments to let parties interpolate the commitment of the secret, and cross-reference it with the other committee, while maintaining secrecy. However, in the lightweight cryptography setting, this approach does not work anymore.

We address this issue and describe our *secret equivalence* protocol, which lets parties in both committees verify whether a dealer has distributed the same secrets, while preserving privacy. In essence, parties reconstruct a random linear combination of the secrets distributed by the dealer among two committees and check whether they are the same. To be more concrete:

1) **Random mask generation**: Each dealer in $\mathsf{P}_{\mathtt{old}}$ dis-

tributes $[s_0]_{t_1}, [s_1]_{t_1}, \ldots, [s_k]_{t_1}$ among parties in $\mathsf{P}_{\mathtt{old}}$ and $[s'_0]_{t_2}, [s'_1]_{t_2}, \ldots, [s'_k]_{t_2}$ among parties in $\mathsf{P}_{\mathtt{new}}$ using $\mathcal{F}_{\mathsf{ACSS\text{-}id}}$. Here, s_0, s'_0 serve as random masks to protect the privacy of secrets.

2) **Coin Generation**: On terminating this dealer's protocol, parties in both $\mathsf{P}_{\mathtt{old}}, \mathsf{P}_{\mathtt{new}}$ agree on a random value r. For this, each party in $\mathsf{P}_{\mathtt{old}}$ generate a common coin r by invoking $\mathcal{F}_{\mathsf{coin}}$, a coin generation functionality, and broadcast it to $\mathsf{P}_{\mathtt{new}}$. We explain the coin generation protocol in the next subsection.
3) **Reconstructing the Random Linear Combination**: Parties in $\mathsf{P}_{\mathtt{old}}$ and $\mathsf{P}_{\mathtt{new}}$ compute shares of $s = s_0 + \sum_{i\in 1}^{k} r^i \cdot s_i$ and $s' = s'_0 + \sum_{i\in 1}^{k} r^i \cdot s'_i$, respectively. Then, they use OEC to reconstruct s, s', broadcast them to the other committee, and check whether $s = s'$. If true, parties in both committees output their shares.

In case $s_i \neq s'_i$ for any $i \in \{1, \ldots, L\}$, then according to the Schwartz-Zippel lemma, the only way that $s = s'$ is if r is a zero of the degree $\leq L$ polynomial, which can only happen with negligible probability of $\leq \frac{L}{2^\kappa}$.

Since a dealer uses $\mathcal{F}_{\mathsf{ACSS\text{-}id}}$ and not ACSS to distribute shares, OEC might not terminate for any honest party in either committee. However, this will not cause a liveness problem because we set the success of this equivalence protocol as a condition for a dealer to be proposed into the ACS of successful dealers. As there are at least $n_1 - t_1$ honest dealers, the random sharing generation will succeed. Note that a successful secret equivalence protocol does not imply all honest parties will eventually have their shares.

2.3.4. Random Coin Generation. Based on the above ideas, all parties need to generate around $\mathcal{O}(n)$ random coins for use in the secret equivalence protocol and the asynchronous BA protocols. We use the HashRand protocol [7] for coin generation, which can generate one random coin with $\mathcal{O}(\kappa n^2 \log(n))$ communication bits. Even though this protocol has a startup latency of $\mathcal{O}(\log(n))$ rounds, it does not impact the overall complexities of our DPSS protocol. Additionally, for the generating random challenge r in the secret equivalence protocol, we use the Fiat-Shamir heuristic with RO. In essence, this approach lets honest parties generate a random challenge r locally by hashing the commitment vector used in the ACSS-Id protocol. We discuss this optimization in Appendix B.

3. Preliminaries

We denote the computational security parameter by κ, and require the field size to be $2^{\Omega(\kappa)}$.

3.1. Shamir Secret Sharing

In this work, we will use the standard Shamir Secret Sharing Scheme [47]. Let n be the number of parties and $\mathbb{F}$ be a finite field of size $|\mathbb{F}| \geq n+1$. Let $\alpha_1, \ldots, \alpha_n$ be n distinct non-zero elements in $\mathbb{F}$.

A *degree-d* Shamir sharing of $x \in \mathbb{F}$ is denoted as $[x]_d$, which is a vector $(x_1, \ldots, x_n)$ and there exists a polynomial $f(\cdot) \in \mathbb{F}[X]$ of degree at most d such that $f(0) = x$ and $f(\alpha_i) = x_i$ for $i \in [n]$. Each party P_i's shares of $[x]_d$ is x_i. The Shamir sharing satisfies linear homomorphism property:

$$\forall\ [x]_d, [y]_d, a, b,\ \ [ax + by]_d = a[x]_d + b[y]_d.$$

3.2. Building Blocks

Our work relies on the following building blocks, which we introduce in more detail in Appendix A.

- $\mathcal{F}_{\mathsf{rbc}}$**.** It denotes reliably broadcast and allows parties to agree on the value of a sender without requiring termination if the sender is corrupted. [24] gives a construction with $\mathcal{O}(Ln + \kappa n^2)$ communication bits for broadcasting L bits messages in the random oracle model.
- $\mathcal{F}_{\mathsf{ba}}$**.** It denotes Byzantine agreement and allows parties to agree on a common message. Multi-valued Byzantine agreement with $\mathcal{O}(L \cdot n + \kappa n^2 \log(n))$ communication bits can be achieved in the random oracle model and given common coins, where L is the size of the message (see [41]). The expected round complexity is $\mathcal{O}(1)$.
- $\mathcal{F}_{\mathsf{mvba}}$**.** It denotes Multi-valued validated Byzantine agreement and allows all parties to agree on a common output that satisfies a fixed external validity predicate. By assuming common coins, the authors in [14] give a construction of $\mathcal{F}_{\mathsf{mvba}}$ with $\mathcal{O}(n \log(n) L + \kappa n^2 \log(n))$ communication and $\log(n)$ rounds, where L is the size of the input message. We also use the protocol in [35] which achieves $\mathcal{O}(nL + \kappa n^2 \log(n))$ with constant rounds, but for sub-optimal resilience of $t < (\frac{1}{3} - \epsilon)n$.
- $\mathcal{F}_{\mathsf{acs}}$**.** It denotes agreement on a common set and allows parties to agree on a set of at least $n - t$ parties that satisfy a certain property. [22] gives a construction with communication complexity $\mathcal{O}(\kappa n^3)$ in the random oracle model.
- $\mathcal{F}_{\mathsf{coin}}$**.** It denotes the random coin and allows all parties to agree on a random value. [7] gives a construction with communication complexity $\mathcal{O}(\kappa n^2 \log n)$ in the random oracle model.

4. Preparation of Random Sharings

Here, we describe our protocol Π_{RandSh}, which prepares random degree-t_1 and t_2 Shamir sharings $[r]_{t_1}, [r]_{t_2}$ with the same secret for all parties in P_1 and P_2 respectively. The communication complexity is $\mathcal{O}(Ln)$ field elements plus $\mathcal{O}(\kappa n^3 \log^2 n)$ bits to prepare L random sharings.

First, we let each party in P_1 act as the dealer and distribute random sharings with the same secret to both committees, and all parties will verify the correctness of sharings distributed by this dealer. For a corrupted dealer, the verification process may never terminate because there is no guarantee that all parties can reconstruct the random linear combination result. This will not affect the termination of Π_{RandSh} because all parties can always verify the sharings distributed by honest dealers.

Protocol Π_{RandSh}: Random Sharing Generation

Public Inputs: $(\alpha_0, \ldots, \alpha_n), L$.

1: **Distribute Random Sharings**: Let $L' = L/(t_1+1)$, for each $\ell \in [0, L']$, each party $P_i \in \mathsf{P}_1$ randomly samples degree-t_1 and t_2 polynomials $f_\ell^{(i)}(x), g_\ell^{(i)}(x)$ such that $f_\ell^{(i)}(\alpha_0) = g_\ell^{(i)}(\alpha_0)$.

2: Each party $P_i \in \mathsf{P}_1$ acts as dealer and invoke two instances of $\mathcal{F}_{\mathsf{ACSS\text{-}id}}$ to share $\{f_\ell^{(i)}(x)\}_{\ell=0}^{L'}$ and $\{g_\ell^{(i)}(x)\}_{\ell=0}^{L'}$ among parties in P_1 and P_2 respectively.

3: **Verify Secret Equivalence**: When all parties terminate $\mathcal{F}_{\mathsf{ACSS\text{-}id}}$ invoked by a dealer, they send request to $\mathcal{F}_{\mathsf{coin}}$ to get a random challenge r. Then they do the following to do the verification.

(a) Let $f_*^{(i)}(x) := \sum_{\ell=0}^{L'} r^\ell \cdot f_\ell^{(i)}(x)$, all parties in P_1 locally compute their shares. Define $g_*^{(i)}(x)$ in the same way and all parties in P_2 locally compute their shares. Each party that cannot compute his shares will keep silent during the verification for this dealer.

(b) Each party $P_j \in \mathsf{P}_1$ who has $f_*^{(i)}(\alpha_j)$ will forward it to all parties in P_1. When he succeeds in using online error correction to reconstruct the degree-t polynomial $f_*^{(i)}(x)$, he forwards $f_*^{(i)}(\alpha_0)$ to all parties in P_1 and P_2. Parties in P_2 do the same thing for $g_*^{(i)}(\alpha_0)$.

(c) For each party in P_1 and P_2, when he receives $f_*^{(i)}(\alpha_0)$ from t_1+1 distinct parties in P_1, $g_*^{(i)}(\alpha_0)$ from t_2+1 distinct parties in P_2, and $f_*^{(i)}(\alpha_0) = g_*^{(i)}(\alpha_0)$, he accepts his shares distributed by this dealer.

Then all parties continue to agree on the set of successful dealers, and extract the random sharings accordingly. Note that some parties may not be able to compute their shares due to corrupted dealers, and they will set their output to $\perp$ and terminate.

Protocol Π_{RandSh}: Random Sharing Generation

1: **Agree on successful dealers**: All parties in P_1 and P_2 do the following to agree on a set D of successful dealers.

(a) All parties in P_2 send $(\texttt{Support}, P_i)$ to parties in P_1 when they terminate $\mathcal{F}_{\mathsf{ACSS\text{-}id}}$ invoked by D and accept their shares.

(b) All parties in P_1 sets the property Q as they terminating $\mathcal{F}_{\mathsf{ACSS\text{-}id}}$ invoked by P_i and accept their shares and receive $(\texttt{Support}, P_i)$ from t_2+1 distinct parties in P_2. Then all parties in P_1 invoke $\mathcal{F}_{\mathsf{acs}}$ with property Q to agree on a set D of size $2t_1+1$.

(c) When a party in P_1 gets D, he forwards it to all parties in P_2. For each party in P_2, when he receives the set D from t_1+1 distinct parties in P_1, he accepts it.

2: **Extract random sharings**: All parties agree on a Vandermonde matrix $\boldsymbol{V}$ of size $(t_1+1) \times (2t_1+1)$. Then for each $\ell \in [L']$, let:

$$\{[f_{\ell,k}(\alpha_0)]_{t_1}\}_{k=1}^{t_1+1} := \boldsymbol{V} \cdot \{[f_\ell^{(j)}(\alpha_0)]_{t_1}\}_{j\in\mathsf{D}}$$
$$\{[g_{\ell,k}(\alpha_0)]_{t_2}\}_{k=1}^{t_1+1} := \boldsymbol{V} \cdot \{[g_\ell^{(j)}(\alpha_0)]_{t_2}\}_{j\in\mathsf{D}}.$$

All parties in P_1 and P_2 locally compute and terminate with their shares of $[f_{\ell,k}(\alpha_0)]_{t_1}$ and $[g_{\ell,k}(\alpha_0)]_{t_2}$ for all $\ell \in [L'], k \in [t_1+1]$ respectively.

4.1. Costs Analysis of Π_{RandSh}

We analyze the communication complexity, round complexity, and computational complexity of our protocol Π_{RandSh} as follows.

Communication Complexity. Our construction of Π_{RandSh} is built based on the ACSS-Id scheme, for each instance of $\mathcal{F}_{\mathsf{ACSS\text{-}id}}$, it can be realized with $\mathcal{O}(Nn)$ field elements plus $\mathcal{O}(\kappa n^2 \log^2 n)$ communication bits [8] for distributing N degree-t Shamir secret sharings. Note that the original report communication cost of $\mathcal{F}_{\mathsf{ACSS\text{-}id}}$ in [8] is $\mathcal{O}(Nn)$ field elements plus $\mathcal{O}(\kappa n^3)$ communication bits, but the additive overhead can easily improve to $\mathcal{O}(\kappa n^2 \log^2 n)$ by replacing their secret sharing procedure by the construction in [49] and the distributed zero-knowledge proof in [13].

Each party will invoke an instance of $\mathcal{F}_{\mathsf{ACSS\text{-}id}}$ to distribute $L' = \mathcal{O}(L/n)$ random sharings, and there are n dealers, then the total communication costs is $\mathcal{O}(Ln)$ field elements plus $\mathcal{O}(\kappa n^3 \log^2 n)$ bits. For the verification of each dealer, all parties only exchange one field element, which causes $\mathcal{O}(n^3)$ field elements in total. Then all parties invoke $\mathcal{F}_{\mathsf{acs}}$ to agree on the set of successful dealers, according to the construction of ACS protocol in [22], it costs $\mathcal{O}(\kappa n^3)$ bits.

Therefore, the total communication complexity is $\mathcal{O}(Ln)$ field elements plus $\mathcal{O}(\kappa n^3 \log^2 n)$ bits, plus the generation of $\mathcal{O}(n)$ random coins.

Round Complexity. Since all parties can invoke all instances of $\mathcal{F}_{\mathsf{ACSS\text{-}id}}$ in parallel, and the expected round complexity of $\mathcal{F}_{\mathsf{ACSS\text{-}id}}, \mathcal{F}_{\mathsf{acs}}$ is constant, then the expected round complexity of Π_{RandSh} is $\mathcal{O}(1)$, plus the round complexity of $\mathcal{F}_{\mathsf{coin}}$ to generate random challenges.

5. Party Elimination-Based Public Reconstruction

In this section, we give of construction of the party elimination-based public reconstruction protocol $\Pi_{\mathsf{PubRec\text{-}PE}}$. The communication complexity is $\mathcal{O}(Ln)$ field elements plus $\mathcal{O}(\kappa n^2 \log n)$ bits to reconstruct L secrets. We start by introducing the sub-protocols $\Pi_{\mathsf{BatchPubRec}}, \Pi_{\mathsf{Agreement}}$ as follows. In the following, we use t to denote the corruption threshold.

Batch Reconstruction. The protocol $\Pi_{\mathsf{BatchPubRec}}$ is used to let all parties reconstruct L secrets in a batch with amortized linear communication costs, and we use the same construction as in [8]. Each party that does not his input shares will broadcast an accusation outside the protocol $\Pi_{\mathsf{BatchPubRec}}$.

Protocol $\Pi_{\mathsf{BatchPubRec}}$

For each party P_i**:**

1: All parties divide the L sharings into $L/(t+1)$ groups, each of size $t+1$. For all groups, all parties do the following things in parallel.

(1). Let $[s^{(0)}]_t, \ldots, [s^{(t)}]_t$ denote the degree-t Shamir secret sharings in each group. Define

$$f(X) = s^{(0)} + s^{(1)} \cdot X + \cdots + s^{(t)} \cdot X^t$$

P_i sends his share of $[f(\alpha_j)]_t$ to each P_j.

(2). To reconstruct $f(\alpha_i)$, P_i waits to receive messages:

- If P_i first succeeds in using online error correction to reconstruct $f(\alpha_i)$, he sends $f(\alpha_i)$ to all parties and proceed.
- Otherwise, if P_i first receives $(\texttt{Accusation}, P_j, D)$ from P_j's reliably broadcast, he records this accusation, and moves to Step 2.

(3). To reconstruct $f(X)$, P_i waits to receive messages from all parties:

- If P_i first succeeds in using online error correction to reconstruct $f(X)$, he records $(s^{(0)}, \ldots, s^{(t+1)})$.
- Otherwise, he does the same thing as in Step 2-(2) (the "otherwise" case) and moves to Step 2.

2: If P_i reconstructs all secrets, he outputs them. Otherwise, he outputs an accusation he received during the reconstruction.

Agreement on Public Reconstruction Result. The protocol $\Pi_{\mathsf{Agreement}}$ is used to let all parties agree on the same result of public reconstruction. Compared to the construction in [8], we reduce the additive communication overhead from cubic to sub-cubic.

Protocol $\Pi_{\mathsf{Agreement}}$

1: All parties invoke $\mathcal{F}_{\mathsf{ba}}$ and party P_i uses the public reconstruction result as his input. Upon receiving the agreement result from $\mathcal{F}_{\mathsf{ba}}$, P_i checks whether it is the same as his input. If true, he sets $b_i = 1$. Otherwise, he sets $b_i = 0$.

2: All parties invoke $\mathcal{F}_{\mathsf{ba}}$ and party P_i uses b_i as his input. Upon receiving the result b from $\mathcal{F}_{\mathsf{ba}}$, if $b = 1$, P_i outputs his public reconstruction. Otherwise, P_i uses $\perp$ as his output.

3: When the output is not $\perp$, all parties terminate. Otherwise, all parties follow the steps to agree on the identity of a corrupted party.

(1). All parties jointly invoke an instance of $\mathcal{F}_{\mathsf{mvba}}$. Each party waits until he receives an accusation from a party's broadcast, and uses the first one he receives as the input for $\mathcal{F}_{\mathsf{mvba}}$. For one party's input, which is in the form of $(\texttt{Accusation}, P_i, D)$, it is validated if all parties receive this accusation from P_i's reliably broadcast.

(2). Upon terminating $\mathcal{F}_{\mathsf{mvba}}$ with output $(\texttt{Accusation}, P_i, D)$, all parties send request $(\texttt{Open-Proof}, P_i)$ to $\mathcal{F}_{\mathsf{ACSS\text{-}id}}$ invoked by D, and terminates with the identity of corrupted they received from $\mathcal{F}_{\mathsf{ACSS\text{-}id}}$.

Finally, we give the construction of $\Pi_{\mathsf{PubRec\text{-}PE}}$ as follows. Note that step 1 is only executed once when all parties first invoke $\Pi_{\mathsf{PubRec\text{-}PE}}$.

Protocol $\Pi_{\mathsf{PubRec\text{-}PE}}$: Public Reconstruction

Public Inputs: L, t

1: **Accusation to Corrupted Dealer.** All parties take their shares of $[s_1]_t, \ldots, [s_L]_t$ as inputs. Each party P_i who does not have shares reliably broadcasts $(\texttt{Accusation}, P_i, D)$ for all active corrupted dealers D know to him. This step is only executed by all parties once.

2: **Public Reconstruction with Party Elimination.** All parties execute $\Pi_{\mathsf{BatchPubRec}}$ with their input shares.

3: All parties jointly invoke $\Pi_{\mathsf{Agreement}}$, each party takes his output of $\Pi_{\mathsf{BatchPubRec}}$ as input. When they terminate

- If the output is the identity of a corrupted dealer, all parties send request $\texttt{Public-Recon}$ to $\mathcal{F}_{\mathsf{ACSS\text{-}id}}$ invoked by this dealer and learn the secrets. Then all parties locally update their shares of $[s_1]_t, \ldots, [s_L]_t$ and terminate.
- Otherwise, all parties terminate with the output.

5.1. Costs Analysis of $\Pi_{\mathsf{PubRec\text{-}PE}}$

We analyze the communication complexity, round complexity, and computational complexity of our protocol $\Pi_{\mathsf{PubRec\text{-}PE}}$ as follows.

Communication Complexity. Let L denote the number of sharings to be reconstructed. At the beginning of $\Pi_{\mathsf{PubRec\text{-}PE}}$, all parties reliably broadcast the set of parties they want to accuse, which requires $\mathcal{O}(\kappa \cdot n^3)$ bits. Then they execute the sub-protocols $\Pi_{\mathsf{BatchPubRec}}, \Pi_{\mathsf{Agreement}}$:

- For $\Pi_{\mathsf{BatchPubRec}}$, all parties exchange $\mathcal{O}(L/(t+1))$ field elements, resulting in $\mathcal{O}(Ln + n^2)$ field elements in total.
- For $\Pi_{\mathsf{Agreement}}$, all parties first invoke two instances of $\mathcal{F}_{\mathsf{ba}}$, according to the construction of $\mathcal{F}_{\mathsf{ba}}$ in [41], it requires $\mathcal{O}(Ln)$ field elements plus $\mathcal{O}(\kappa n^2 \log n))$ bits plus $\mathcal{O}(1)$ random coin.
- During the $\Pi_{\mathsf{Agreement}}$, if all parties continue to trigger the agreement process of corrupted parties, they additionally invoke an instance of $\mathcal{F}_{\mathsf{mvba}}$. It requires $\mathcal{O}(\kappa n^2 \log n))$ bits plus $\mathcal{O}(\log \kappa)$ random coin. After agreeing on an accusation from $\mathcal{F}_{\mathsf{mvba}}$, they will request $\mathcal{F}_{\mathsf{ACSS\text{-}id}}$ to learn the identity of a corrupted party. Let N denote the number of secrets distributed by one party by $\mathcal{F}_{\mathsf{ACSS\text{-}id}}$, according to [8], [49], this step requires $\mathcal{O}(Nn)$ field elements plus $\mathcal{O}(\kappa n^2 \log n)$ bits. All parties also request $\mathcal{F}_{\mathsf{ACSS\text{-}id}}$ to reconstruct the secrets shared by the corrupted party, which costs $\mathcal{O}(Nn + n^2)$ field elements.

Therefore, the communication complexity of $\Pi_{\mathsf{PubRec\text{-}PE}}$ is $\mathcal{O}(Ln)$ field elements plus $\mathcal{O}(\kappa n^2 \log n)$ bits. If all parties trigger the agreement process of corrupted parties, it additionally costs $\mathcal{O}(Nn)$ field elements plus $\mathcal{O}(\kappa n^2 \log n)$ bits. All parties also need to prepare $\mathcal{O}(\log \kappa)$ random coins, which will be used for $\mathcal{F}_{\mathsf{ba}}, \mathcal{F}_{\mathsf{mvba}}$.

Round Complexity. If all parties do not trigger of agreement of corrupted parties, the round complexity is the same as $\mathcal{F}_{\mathsf{ba}}$, which is expected constant according to [41]. If they trigger the agreement of corrupted parties, according to [27], the expect round complexity of $\mathcal{F}_{\mathsf{mvba}}$ is $\mathcal{O}(\log \kappa)$, then the total round complexity of $\Pi_{\mathsf{PubRec\text{-}PE}}$ is $\mathcal{O}(\log \kappa)$.

6. Asynchronous Dynamic Proactive Secret Sharing

6.1. Construction of DPSS

In this section, we introduce the detailed construction of our DPSS protocol. The communication complexity is $\mathcal{O}(Ln)$ field elements plus $\mathcal{O}(\kappa n^3 \log^2 n)$ to transfer L secrets. For security, we prove lemma 1 in Appendix C.

Protocol Π_{DPSS}: DPSS protocol

Public Inputs: L.
All parties in P_1 take their shares of $[s_\ell]_{t_1}$ for $\ell \in [L]$ as inputs. When they receive request $(\texttt{Transfer}, \mathsf{P}_2)$ from the environment, they do the following.

1: **Generate Random Sharings**: Parties in both committees participate in Π_{RandSh} to generate random Shamir sharings $[r_\ell]_{t_1}$ and $[r_\ell]_{t_2}$ for parties in P_1 and P_2 respectively.

2: **Blinding Secrets**: Parties in P_1 locally compute their shares of $[b_\ell]_{t_1} = [s_\ell]_{t_1} + [r_\ell]_{t_1}$ for all $\ell \in [L]$.

3: **Reconstruct Blinded Secrets**: All parties in P_1 divide their shares of $\{[b_\ell]_{t_1}\}_{\ell=1}^{L}$ into t_1 segments, then they invoke $\Pi_{\mathsf{PubRec\text{-}PE}}$ for each segment in order and take their shares in this segment as inputs.

4: **Shares Transfer**: Let $b_* = b_1 || \cdots || b_L$, all parties in P_1 divide b_* into $t_2 + 1$ parts $b_*^{(0)}, \ldots, b_*^{(t)}$ and define $B(x)$ as follows:

$$B(X) := b_*^{(0)} + b_*^{(1)} \cdot X + \cdots + b_*^{(t_2)} \cdot X^{t_2}$$

Then each party $P_i \in \mathsf{P}_1$ sends $B(\alpha_i)$ to parties in P_2 and terminates. Parties in P_2 use online error correction to recover $B(x)$ and get $b_1, \ldots, b_L$. Then for each party in P_2, if he has shares of $[r_\ell]_{t_2}$, he terminates with his shares of $[s_\ell]_{t_2} = b_\ell - [r_\ell]_{t_2}$ for all $\ell \in [L]$. Otherwise, he terminates with the symbol `success`.

Lemma 1. Protocol Π_{DPSS} UC-securely computes $\mathcal{F}_{\mathsf{DPSS}}$ in the $\{\mathcal{F}_{\mathsf{mvba}}, \mathcal{F}_{\mathsf{acs}}, \mathcal{F}_{\mathsf{ACSS\text{-}id}}, \mathcal{F}_{\mathsf{coin}}\}$-hybrid model against a fully malicious adversary $\mathcal{A}$ who corrupts at most $t < n/3$ parties in the asynchronous setting.

6.2. Recovery of Shares

If all parties in P_2 need their shares of output for other tasks, they jointly invoke the following protocol Π_{Rec}. In our construction, each honest party who can not compute his shares will first accuse the corrupted dealer known by him, then wait for all parties to agree on the identity of this dealer, and finally get the secrets reconstructed by all parties. Then all parties locally update their shares with the secrets.

The issue here is all parties will never know whether all honest parties get their shares, so they will take a common termination condition as input, which is determined by the outside task and used to ensure the termination of Π_{Rec}. The communication complexity of Π_{Rec} is $\mathcal{O}(Ln)$ field elements plus $\mathcal{O}(\kappa n^3 \log n)$ bits.

Protocol Π_{Rec}: Recover Process

Public Inputs: Common termination condition T.

1: Upon receiving `RecoverSh` from the environment, all parties keep online and do the following:
- For each party P_i who cannot compute his shares of $[r_\ell]_{t_2}$, he reliably broadcasts $(\texttt{Accusation}, P_i, D)$ in P_2 for all active corrupted dealers D known by him.

2: All parties invoke an instance of $\mathcal{F}_{\mathsf{mvba}}$. For each party P_i, if he has broadcast $(\texttt{Accusation}, P_i, D)$ for some D, he will take this message as input for $\mathcal{F}_{\mathsf{mvba}}$ (for only one D). Otherwise, P_i will wait to receive messages from others, and use the first $(\texttt{Accusation}, P_k, D)$ he received from P_k's reliably broadcast as his inputs for $\mathcal{F}_{\mathsf{mvba}}$.

3: When all parties terminate $\mathcal{F}_{\mathsf{mvba}}$ with an output $(\texttt{Accusation}, P_k, D)$, they send request $(\texttt{Open-Proof}, P_k)$ to $\mathcal{F}_{\mathsf{ACSS\text{-}id}}$ invoked by D and gets output $(\texttt{Corrupt}, P_k)$ or $(\texttt{Corrupt}, D)$.

4: Upon agreeing on the identity of the corrupted party, all parties send `Public-Recon` to $\mathcal{F}_{\mathsf{ACSS\text{-}id}}$ invoked by him and get the secrets. Then all parties locally update their shares of $[r_\ell]_{t_2}$ and then compute their shares of $[s_\ell]_{t_2} = b_\ell - [r_\ell]_{t_2}$ for all $\ell \in [L]$.

5: All parties check termination condition T, if true, they terminate. Otherwise, they move to Step 1.

6.3. Costs Analysis of Π_{DPSS} and Π_{Rec}

We analyze the communication complexity, round complexity, and computational complexity of our protocol $\Pi_{\mathsf{DPSS}}, \Pi_{\mathsf{Rec}}$ as follows.

Communication Complexity. For Π_{DPSS}, all parties first invoke Π_{RandSh} to generate L random degree-t Shamir sharings, which requires $\mathcal{O}(Ln)$ field elements plus $\mathcal{O}(\kappa n^3 \log^2 n)$ bits. Note that during Π_{RandSh}, each party will distribute $\mathcal{O}(L/n)$ random sharings.

Then they divide the public reconstruction of blinded secrets into $\mathcal{O}(n)$ segments, then invoke $\Pi_{\mathsf{PubRec\text{-}PE}}$ for each segment in sequence, the communication for parties not trigger the agreement of corrupted parties during $\Pi_{\mathsf{PubRec\text{-}PE}}$ will be $\mathcal{O}(n \cdot (\frac{L}{n} + 1) \cdot n) = \mathcal{O}(Ln + n^2)$ field elements plus $\mathcal{O}(n \cdot \kappa n^2 \log n) = \mathcal{O}(\kappa n^3 \log n)$ bits. For the additional costs when all parties trigger the agreement of corrupted parties, it will also cost $\mathcal{O}(Ln + n^2)$ field elements plus $\mathcal{O}(\kappa n^3 \log n)$ bits.

Finally, parties in P_1 transfer the blinded secrets to parties in P_2, which requires $\mathcal{O}(Ln + n^2)$ field elements.

Therefore, the total communication complexity of Π_{DPSS} is $\mathcal{O}(Ln)$ field elements plus $\mathcal{O}(\kappa n^3 \log^2 n)$ bits. The cost of Π_{Rec} is similar to the analysis of Step 3 of Π_{DPSS}, and is $\mathcal{O}(Ln)$ field elements plus $\mathcal{O}(\kappa n^3 \log n)$ bits. All parties

also need to prepare $\mathcal{O}(n \log \kappa)$ random coins during the whole process.

Round Complexity. The round complexity of Π_{RandSh} is $\mathcal{O}(1)$ and all parties only invoke it for once. The round complexity of $\Pi_{\mathsf{PubRec\text{-}PE}}$ is $\mathcal{O}(\log \kappa)$ and it will be invoked for $\mathcal{O}(n)$ times, where t is the number of corrupted parties. Therefore, the total round complexity is $\mathcal{O}(n + t \log \kappa)$.

7. Evaluation

In this evaluation section, we aim to answer the following questions.

1) **Performance**: How well does our protocol perform compared to existing works?
2) **Scalability**: How does our protocol scale with increasing system size n and load L?

In addition, we also aim to discuss the impact of our design choices over three dimensions: Computation, Communication, and Round complexity. We design our experiments to analyze the impact of each dimension over the overall protocol performance.

1) **Computation**: We evaluate the impact of computational efficiency of our protocol by comparing our performance to that of Long Live [55], which uses public-key cryptography.
2) **Communication**: We compare the performance of our linear communication cost protocol with a naive quadratic cost variant where parties execute the framework described in Section 2.2 with the quadratic cost ACSS protocol of Shoup and Smart [49].
3) **Rounds**: We compare the performance of our protocol in two situations: (a) An optimistic fault-free constant-round version that runs reconstructs all secrets in one segment, (b) Our full linear cost protocol with $t = \frac{n-1}{3}$ malicious parties in both committees. In addition, we implement our full protocol with a round-efficient partially synchronous BA protocol [40] to understand and reflect its impact in contemporary blockchains, which implement partially synchronous protocols.

7.1. Implementation and Testbed Setup

We provide an end-to-end implementation of our DPSS protocols in Rust [2]. We use the `tokio` library as asynchronous runtime. We utilize a Fast Fourier Transform friendly 252-bit prime field for our protocol. We realize finite-field arithmetic using the Lambdaworks Rust library and utilize Fast-Fourier transforms for share preparation wherever possible.

Protocol Choices. Our DPSS implementation employs multiple sub protocols, which we describe here. For Reliable Broadcast, we implemented Cachin-Tessaro's RBC protocol [10] with Reed-Solomon Erasure codes, which are highly computationally efficient. We also implemented a batched version of the AVID protocol in DispersedLedger [53] for our ACSS-Id protocol, again mainly driven by the computational efficiency of Erasure Codes.

For asynchronous consensus, we implemented Das *et al* 's Hash-based ACS protocol [22]. This ACS protocol does not require a DKG or PKI setup and uses only lightweight cryptography to achieve consensus with $\mathcal{O}(\kappa n^3)$ complexity. However, since this protocol has a high round complexity, we only utilize it in the first consensus instance to generate random sharings. For all subsequent invocations of MVBA, we adopted the implementation [6] of FIN MVBA [26], which utilizes common coins generated in the first consensus instance. Although this protocol has $\mathcal{O}(\kappa n^3)$ additive overhead, its relative simplicity and lower round complexity led us to make this choice.

Based on these building blocks and in line with our experiment design, we implement the following protocols.

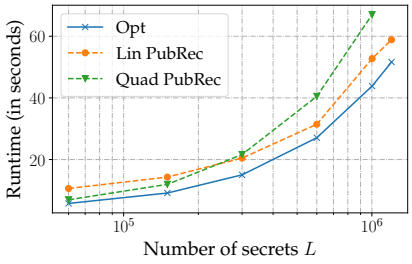
1) An optimistic linear-cost fault-free constant-round version that reconstructs all secrets in a single segment.
2) A quadratic cost constant-round DPSS protocol that can tolerate malicious parties. This protocol uses Shoup and Smart [49] with the framework described in Section 2.2.
3) Our full linear cost protocol.
4) Our full linear cost protocol with partially synchronous consensus.

Commitment Generation for Long Messages We employ a cryptographic Hash function for Random Oracle invocations. In our DPSS protocol, we invoke the Hash function on long messages of length $O(L)$. However, the computational cost of a Hash function like `SHA256` increases linearly with input length. In addition, CPU-based implementation of `SHA256` is slow, with a 10 KB message taking 38 microseconds for Hashing.

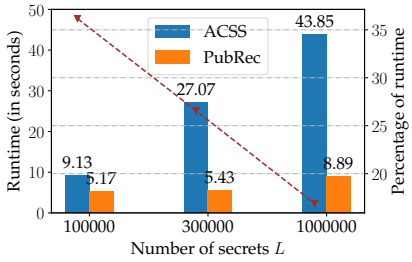
We reduce this cost by employing a Hash function built using the AES symmetric-key cipher [45]. The main advantage of this function is that we employ hardware-accelerated variant of AES available as the AES-NI instruction set in almost all modern processors. However, this construction only offers a fixed compression ratio of 2 i.e. it compresses an input of size 512 bits to 256 bits. So, to hash a long message, we create a Merkle tree of this message and compute the root of this tree as the message's Hash. This construction takes only takes 11 microseconds to hash a 10 KB message, resulting in a $3.4\times$ speedup compared to CPU-based `SHA256`.

Evaluation setup We set up a geo-distributed testbed on Amazon Web Services (AWS). We evaluate the time taken by the protocol to transfer shares of L secrets between two committees of equal sizes n. We evaluate with varying numbers of parties $n = 16, 64, 112$ within each committee. We run our protocol on `c5.large` nodes, each with 2 cores and 8 GB RAM. We divide each machine between two parties and provision resources such that each party in either committee has 1 CPU core and 4 GB of RAM to execute the protocol. We create a geo-distributed testbed of n nodes to simulate

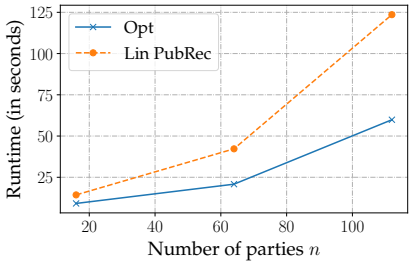
2. Available at https://github.com/akhilsb/acss-rs



(a) **Runtime vs Secrets** L: With $n = 16$ parties in both committees. With increasing L, linear cost protocol becomes more efficient, crossing over the quadratic cost protocol at $L = 200000$, showcasing the advantage of linear communication cost at scale. The gap between our linear and optimistic protocols stays constant.



(b) **Runtime decomposition of linear-cost DPSS**: With $n = 16$ and $t = 5$ parties in both committees. The dotted line plots the percentage of runtime consumed by the PubRec procedure. With increasing L, this share decreases, indicating the diminishing effect of the high round complexity.



(c) **Scalability plot**: For $L = 100000$ secrets. At $n = 112$ parties, our optimistic case execution ($t = 0$) runs with a minute and our linear cost pessimistic case ($t = 37$) within two minutes. The gap between best and worst cases also increases with n because of the higher round complexity of our linear cost protocol.

Figure 2: Runtime evaluation of our DPSS protocols. The evaluation assumes two committees with n parties each. Solid line plots indicate optimistic or best case executions with no corrupt parties. Dashed lines indicate worst case executions with $t = \frac{n-1}{3}$ corrupt parties.

execution over the internet. We distribute the machines and parties equally across 8 regions: `N. Virginia, Ohio, N. California, Oregon, Canada, Ireland, Singapore, Tokyo`.

Evaluation metrics We choose the latency of termination as the sole metric for evaluating protocols' performances. Concretely, we provision a central time keeping machine called the *synchronizer* that issues `Start` commands to parties. Once a party terminates the protocol, it sends a `Terminated` command to the synchronizer, which records the latency of each party and reports the average latency of all parties.

Unlike long-running services such as blockchains, where varying request loads necessitate latency-throughput curves for benchmarking, DPSS protocols are invoked only once per epoch or committee change. Their primary goal is to transfer a large batch of secrets efficiently, making termination latency the most relevant performance metric.

7.2. Comparison to Prior Works

We compare the full version of our protocol (variant (c)) with the optimistic version of Long Live [55], which uses KZG commitments and public key encryptions. We configure our protocol with n parties and $t = \frac{n-1}{3}$ malicious parties in both committees. Long Live's implementation runs all n processes in a single core machine. So, to ensure a fair comparison, we evaluated our protocol and Long Live on a single core instance on AWS. We ran DPSS with $n = 16$ parties in both committees for both protocols. For sharing a batch of $L = 10000$ secrets, our protocol took 10.94 seconds compared to Long Live's 241.57 seconds, indicating a 22.1× improvement. We also tried running Long Live for higher values of L and n, but were unsuccessful.

We also tried comparing our work with Cobra [51]. However, we were not able to run their implementation, mainly because of runtime errors. Therefore, we compare our results with the results reported in their paper. We ensure a fair comparison among protocols by making sure that we evaluate our protocols with lower resources per party. The authors in Cobra allocated 28 CPU cores and 896 GB RAM among 14 parties (7 in both committees), which comes to 2 cores and 16 GB of RAM per party. We compare our $n = 16$ evaluation over the described geo-distributed testbed, where each party has 1 CPU core and 4 GB of RAM, with Cobra's numbers. For a batch size of $L = 100000$ secrets, Cobra took 743.1 seconds, compared to 14.31 seconds for our protocol, indicating a 50× speed up.

These improvements over both prior works approximately reflect the two orders of magnitude speedup offered by lightweight cryptographic tools over Discrete-Log cryptography. Furthermore, these results also validate our design principle of prioritizing computational efficiency for scalability and performance.

7.3. Impact of Linear Communication Cost

We explore the impact of linear communication cost on performance and scalability. We evaluate the first three variants, the first with no malicious parties (optimistic) and second and third (pessimistic) with $t = \frac{n-1}{3}$ malicious parties, and report the results in Fig. 2. In the first subplot in Fig. 2a, we ran the three variants with varying L with $n = 16$ parties in each committee.

On comparing both pessimistic cases, we notice that for small L, the quadratic variant outperforms the linear cost protocol. This advantage can be attributed to the linear round complexity and high round trip time on the geo-distributed network, which penalizes the linear cost protocol at low

L. However, as L increases, the linear PubRec protocol narrows the gap and eventually outperforms the quadratic cost protocol after $L = 300000$ secrets. Additionally, at $L > 500000$, we see that the quadratic cost protocol substantially underperforms, mainly because of the high communication overhead per secret.

Scalability: Furthermore, we also note that this crossover point between these two protocols occurs at lower values of L as n increases, and the gap widens substantially faster. For instance, at $n = 64$, the crossover point occurs at $L = 100000$ secrets. At $L = 200000$ secrets, the linear cost protocol takes only 46.6 seconds compared to 86.3 seconds for the quadratic cost protocol. The quadratic cost protocol also crashed for higher values of L and n. These results demonstrate the impact of our linear cost protocol on the overall scalability.

7.4. Impact of Linear Round Complexity

We investigate the impact of round complexity on our linear cost protocol. We measure difference between the optimistic case execution (type 1) and the full execution (type 3) in Fig. 2b. This plot contains the runtime decomposition of the pessimistic case execution of our linear cost protocol. At small L, the public reconstruction phase of the linear cost protocol has a larger share of the entire runtime, mainly because of the linear round complexity. However, as L increases, the share of runtime consumed by this phase reduces. This result is also mainly because of the low communication footprint of this phase, driven by our proposed improvements.

Scalability We also report the results of our evaluation at $n = 16, 64, 112$ in Fig. 2c. The optimistic case of our protocol takes 59 seconds to reshare a batch of $L = 100000$ secrets at $n = 112$ parties. This runtime increases to 123 seconds for the pessimistic case. Furthermore, we note that the gap between optimistic and pessimistic cases increases with increasing n. This gap is again driven by higher round complexity at higher n.

7.5. Using Partially Synchronous Consensus

We discuss the evaluation of our DPSS protocols in a partially synchronous network setting as a possible way of improving the performance of our protocol. Typically, asynchronous BA protocols require more roundtrips, which amplifies the impact of round complexity on the overall protocol. We conduct this evaluation to showcase the impact of the round complexity of asynchronous BA on the overall runtime and potential performance benefits of deploying our protocols with partially synchronous consensus. In essence, the entire protocol remains the same, except for replacing asynchronous BA with a partially synchronous consensus protocol. We utilize the Istanbul BFT consensus algorithm [40], which is a modified version of the Tendermint consensus protocol that guarantees agreement among parties. We also evaluate this protocol only in a stable leader setting. We present the results in Fig. 3.

Finally, we conclude by stating that we demonstrated scalability over both dimensions - number of secrets L and the overall number of parties n - and substantially improved the practicality of DPSS.

8. Related Works

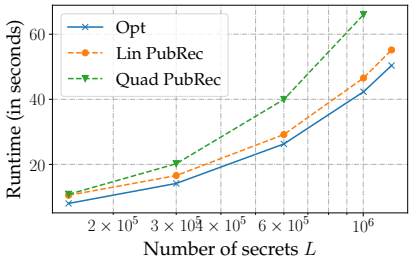
In this section, we discuss the variants of Verifiable Secret Sharing (VSS), then Dynamic Proactive Secret Sharing (DPSS), and protocols with lightweight cryptography.

Verifiable Secret Sharing: VSS protocols in partially synchronous or asynchronous networks with resilience $n = 3t + 1$ can be classified according to their termination property. Asynchronous VSS protocols [4], [5], [9], [24] guarantee the survival or reconstructability of the secret while maintaining privacy against a t-party adversary. On the other hand, ACSS protocols require each party to possess a share of the secret [1], [2], [23], [24], [25], [34], [54], [55]. AVSS protocols are typically easier to realize than ACSS protocols mainly because of relaxed functionality. ACSS protocols are harder to realize and often require primitives like bivariate polynomials [11], [34] or public key encryption [24], [54], [55], which causes them to be more expensive than AVSS protocols. In the lightweight cryptography setting, achieving ACSS requires additional communication of factor $\mathcal{O}(n)$ [49].

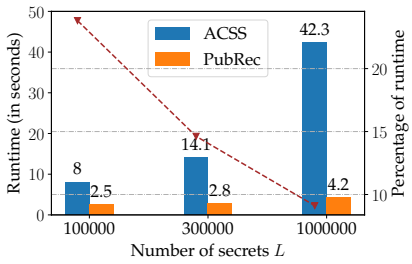
Proactive Secret Sharing: Cachin et al. [11] propose the first asynchronous protocol in this space. This primitive allowed parties to refresh their shares to prevent the secret from being leaked to a mobile adversary who can move across parties in the same system. Proactive secret-sharing schemes can also be classified into two categories, like in VSS protocols: (a) Protocols with reconstructability [46], [51], and (b) protocols with completeness [11], [55]. Cachin *et al* 's protocol first realizes an ACSS protocol with completeness using bivariate polynomials and discrete-log commitments and then uses it to achieve APSS with completeness.

Schultz *et al* [46] relaxed the setting to being dynamic where parties in the committee can also change with epochs. This work formalized the definition of epochs and established conditions of adversarial corruptions that made this primitive possible in non-synchronous environments. Their protocol utilized a partially synchronous consensus mechanism inspired by PBFT to achieve DPSS. Another recent work Cobra [51] uses a HotStuff-style linear consensus mechanism to reduce complexity by a $\mathcal{O}(n)$ factor. Furthermore, both protocols fall under the reconstructability paradigm, where the parties only ensure the survival of the secret.

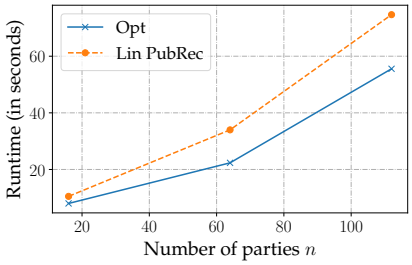
The most recent work titled Long Live the Honey Badger [55] proposed a low- and high-threshold asynchronous DPSS scheme based on an ACSS protocol. This DPSS protocol achieves the stronger definition of completeness, where each party in the new committee also terminates with a share. However, they require a powers of tau ceremony, also called a Structured Random String (SRS) setup, to achieve linear cost.



(a) **Runtime vs Secrets** L: With $n = 16$ parties in both committees. The crossover point between quadratic and linear cost protocols comes much earlier at $L = 100000$ secrets. The gap between optimistic and pessimistic executions is also lower than the asynchronous evaluation.



(b) **Runtime decomposition of linear-cost DPSS**: With $n = 16$ and $t = 5$ parties in both committees. The dotted line plots the percentage of runtime consumed by the PubRec procedure. The reduced percentage of runtime consumed as compared to the asynchronous evaluation.



(c) **Scalability plot**: For $L = 100000$ secrets. At $n = 112$ parties, our optimistic case execution ($t = 0$) runs within 50 seconds and our linear cost pessimistic case ($t = 37$) within 77 seconds. The gap between best and worst cases is substantially lower than in the asynchronous evaluation case.

Figure 3: Runtime evaluation of our DPSS protocols with a partially synchronous consensus protocol.

Lightweight Cryptography. To the best of our knowledge, Backes *et al* [5] showed for the first time that commitment homomorphism as a tool is not necessary to build distributed cryptography by constructing a Hash-based AVSS protocol with $\mathcal{O}(n^2L)$ complexity. Recently Shoup and Smart [49] proposed an ACSS protocol based solely on Hash functions. However, this protocol requires $\mathcal{O}(n^2L)$ communication. This work proposed that protocols based on lightweight cryptography can act as a middle ground between computationally expensive homomorphic cryptography-based and communication-expensive information-theoretic cryptosystems. They also theorized such protocols being the most practical way of achieving Post-Quantum security, especially considering the expensiveness of alternate forms of Post-Quantum cryptography.

Multiple recent works have employed lightweight cryptography for applications like asynchronous consensus [22] and random beacons [7]. These works showed that the speed up gained from computational efficiency far outpaces the minor increases in communication. Recently, a few works even proposed AMPC protocols in this model [8], [39]. These works utilize lightweight cryptography in a new way, in which they force a malicious party to commit its malicious behavior, enabling honest parties to detect it and exclude the party from the system.

9. Conclusion

We propose the first efficient DPSS schemes based on lightweight cryptography (hash functions and symmetric-key encryption). Our solutions achieve optimal resilience and are post-quantum secure.

Compared to prior work, the usage of light cryptographic tools allows for an attractive trade-off in terms of computation and communication, leading to substantially reduced overall latency. In particular, our end-to-end evaluation shows that our protocol is practical for resharing $L = 100000$ secrets among $n = 112$ parties, surpassing the state of the art by a significant margin.

References

[1] Ittai Abraham, Philipp Jovanovic, Mary Maller, Sarah Meiklejohn, and Gilad Stern. Bingo: Adaptivity and asynchrony in verifiable secret sharing and distributed key generation. pages 39–70, 2023.

[2] Nicolas AlHaddad, Mayank Varia, and Haibin Zhang. High-threshold avss with optimal communication complexity. In Nikita Borisov and Claudia Diaz, editors, *Financial Cryptography and Data Security*, pages 479–498, Berlin, Heidelberg, 2021. Springer Berlin Heidelberg.

[3] Shahla Atapoor, Karim Baghery, Daniele Cozzo, and Robi Pedersen. Vss from distributed zk proofs and applications. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 405–440. Springer, 2023.

[4] Michael Backes, Amit Datta, and Aniket Kate. Asynchronous computational vss with reduced communication complexity. In *Topics in Cryptology–CT-RSA 2013: The Cryptographers' Track at the RSA Conference 2013, San Francisco, CA, USA, February 25-March 1, 2013. Proceedings*, pages 259–276. Springer, 2013.

[5] Michael Backes, Aniket Kate, and Arpita Patra. Computational verifiable secret sharing revisited. In *Advances in Cryptology–ASIACRYPT 2011: 17th International Conference on the Theory and Application of Cryptology and Information Security, Seoul, South Korea, December 4-8, 2011. Proceedings 17*, pages 590–609. Springer, 2011.

[6] Akhil Bandarupalli, Adithya Bhat, Saurabh Bagchi, Aniket Kate, Chen-Da Liu-Zhang, and Michael K Reiter. Delphi: Efficient asynchronous approximate agreement for distributed oracles. In *2024 54th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 456–469. IEEE, 2024.

[7] Akhil Bandarupalli, Adithya Bhat, Saurabh Bagchi, Aniket Kate, and Michael K Reiter. Random beacons in monte carlo: Efficient asynchronous random beacon without threshold cryptography. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 2621–2635, 2024.

[8] Akhil Bandarupalli, Xiaoyu Ji, Aniket Kate, Chen-Da Liu-Zhang, and Yifan Song. Computationally efficient asynchronous MPC with linear communication and low additive overhead. Cryptology ePrint Archive, Paper 2024/1666, 2024.

[9] Soumya Basu, Alin Tomescu, Ittai Abraham, Dahlia Malkhi, Michael K Reiter, and Emin Gün Sirer. Efficient verifiable secret sharing with share recovery in bft protocols. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pages 2387–2402, 2019.

[10] C. Cachin and S. Tessaro. Asynchronous verifiable information dispersal. In *24th IEEE Symposium on Reliable Distributed Systems (SRDS'05)*, pages 191–201, 2005.

[11] Christian Cachin, Klaus Kursawe, Anna Lysyanskaya, and Reto Strobl. Asynchronous verifiable secret sharing and proactive cryptosystems. In *ACM CCS 2002*, page 88–97, 2002.

[12] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd Annual Symposium on Foundations of Computer Science, FOCS 2001, 14-17 October 2001, Las Vegas, Nevada, USA*, pages 136–145. IEEE Computer Society, 2001.

[13] Ignacio Cascudo, Daniele Cozzo, and Emanuele Giunta. Verifiable secret sharing from symmetric key cryptography with improved optimistic complexity. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 100–128. Springer, 2025.

[14] Jinyuan Chen. Ociormvba: Near-optimal error-free asynchronous mvba. *arXiv preprint arXiv:2501.00214*, 2024.

[15] Benny Choc, Shafi Goldwasser, Silvio Micali, and Baruch Awerbuch. Verifiable secret sharing and achieving simultaneity in the presence of faults. In *Annual Symposium on Foundations of Computer Science (Proceedings)*, pages 383–395, 1985.

[16] Ashish Choudhury and Arpita Patra. An efficient framework for unconditionally secure multiparty computation. *IEEE Transactions on Information Theory*, 63(1):428–468, 2017.

[17] Ashish Choudhury and Arpita Patra. On the communication efficiency of statistically secure asynchronous mpc with optimal resilience. *Journal of Cryptology*, 36(2):13, 2023.

[18] Ran Cohen. Asynchronous secure multiparty computation in constant time. pages 183–207, 2016.

[19] Sandro Coretti, Juan A. Garay, Martin Hirt, and Vassilis Zikas. Constant-round asynchronous multi-party computation based on one-way functions. pages 998–1021, 2016.

[20] Ivan Damgård and Jesper Buus Nielsen. Scalable and unconditionally secure multiparty computation. In *Advances in Cryptology - CRYPTO 2007, 27th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2007, Proceedings*, volume 4622 of *Lecture Notes in Computer Science*, pages 572–590. Springer, 2007.

[21] George Danezis, Giacomo Giuliari, Eleftherios Kokoris Kogias, Markus Legner, Jean-Pierre Smith, Alberto Sonnino, and Karl Wüst. Walrus: An efficient decentralized storage network, 2025.

[22] Sourav Das, Sisi Duan, Shengqi Liu, Atsuki Momose, Ling Ren, and Victor Shoup. Asynchronous consensus without trusted setup or public-key cryptography. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 3242–3256, 2024.

[23] Sourav Das, Zhuolun Xiang, Lefteris Kokoris-Kogias, and Ling Ren. Practical asynchronous high-threshold distributed key generation and distributed polynomial sampling. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 5359–5376, 2023.

[24] Sourav Das, Zhuolun Xiang, and Ling Ren. Asynchronous data dissemination and its applications. pages 2705–2721, 2021.

[25] Sourav Das, Zhuolun Xiang, Alin Tomescu, Alexander Spiegelman, Benny Pinkas, and Ling Ren. Verifiable secret sharing simplified. *IEEE Security and Privacy 2025*, 2023.

[26] Sisi Duan, Xin Wang, and Haibin Zhang. FIN: Practical signature-free asynchronous common subset in constant time. pages 815–829, 2023.

[27] Hanwen Feng, Zhenliang Lu, and Qiang Tang. õptimal adaptively secure hash-based asynchronous common subset. *Cryptology ePrint Archive*, 2024.

[28] FileCoin. A powerful and dynamic distributed cloud storage network, 2023.

[29] Yossi Gilad, Rotem Hemo, Silvio Micali, Georgios Vlachos, and Nickolai Zeldovich. Algorand: Scaling byzantine agreements for cryptocurrencies. In *Proceedings of the 26th symposium on operating systems principles*, pages 51–68, 2017.

[30] Vipul Goyal, Abhiram Kothapalli, Elisaweta Masserova, Bryan Parno, and Yifan Song. Storing and retrieving secrets on a blockchain. pages 252–282, 2022.

[31] Lov K. Grover. A fast quantum mechanical algorithm for database search, 1996.

[32] Xiaoyu Ji, Junru Li, and Yifan Song. Linear-communication asynchronous complete secret sharing with optimal resilience. In *Annual International Cryptology Conference*, pages 418–453. Springer, 2024.

[33] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. pages 177–194, 2010.

[34] Eleftherios Kokoris Kogias, Dahlia Malkhi, and Alexander Spiegelman. Asynchronous distributed key generation for computationally-secure randomness, consensus, and threshold signatures. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, pages 1751–1767, 2020.

[35] Jovan Komatovic, Joachim Neu, and Tim Roughgarden. Toward optimal-complexity hash-based asynchronous MVBA with optimal resilience. Cryptology ePrint Archive, Paper 2024/1682, 2024.

[36] Aptos Labs. Roll with move: Secure, instant randomness on aptos, 2023.

[37] Junru Li and Yifan Song. Constant-round asynchronous MPC with optimal resilience and linear communication. Cryptology ePrint Archive, Paper 2025/1032, 2025.

[38] Donghang Lu, Thomas Yurek, Samarth Kulshreshtha, Rahul Govind, Aniket Kate, and Andrew Miller. Honeybadgermpc and asynchromix: Practical asynchronous mpc and its application to anonymous communication. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, CCS '19, page 887–903, New York, NY, USA, 2019. Association for Computing Machinery.

[39] Atsuki Momose. Practical asynchronous mpc from lightweight cryptography. *Cryptology ePrint Archive*, 2024.

[40] Henrique Moniz. The istanbul bft consensus algorithm, 2020.

[41] Kartik Nayak, Ling Ren, Elaine Shi, Nitin H Vaidya, and Zhuolun Xiang. Improved extension protocols for byzantine broadcast and agreement. *arXiv preprint arXiv:2002.11321*, 2020.

[42] Arcana Network. Adkg and threshold cryptosystems, 2021.

[43] Torben Pryds Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In *Annual international cryptology conference*, pages 129–140. Springer, 1991.

[44] Supra Research. A decentralized ethos: Distributed key generation and secret sharing, 2023.

[45] Phillip Rogaway and John Steinberger. Constructing cryptographic hash functions from fixed-key blockciphers. In *Annual International Cryptology Conference*, pages 433–450. Springer, 2008.

[46] David Schultz, Barbara Liskov, and Moses Liskov. Mpss: Mobile proactive secret sharing. *ACM Trans. Inf. Syst. Secur.*, 13(4), December 2010.

[47] Adi Shamir. How to share a secret. *Commun. ACM*, 22(11):612–613, November 1979.

[48] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM J. Comput.*, 26(5):1484–1509, oct 1997.

[49] Victor Shoup and Nigel P. Smart. Lightweight asynchronous verifiable secret sharing with optimal resilience. Cryptology ePrint Archive, Paper 2023/536, 2023. https://eprint.iacr.org/2023/536.

[50] Yuan Su, Yuan Lu, Jiliang Li, Yuyi Wang, Chengyi Dong, and Qiang Tang. Dumbo-MPC: Efficient fully asynchronous MPC with optimal resilience. USENIX Security 2025, 2024.

[51] Robin Vassantlal, Eduardo Alchieri, Bernardo Ferreira, and Alysson Bessani. Cobra: Dynamic proactive secret sharing for confidential bft services. In *2022 IEEE symposium on security and privacy (SP)*, pages 1335–1353. IEEE, 2022.

[52] Gavin Wood et al. Ethereum: A secure decentralised generalised transaction ledger. *Ethereum project yellow paper*, 151(2014):1–32, 2014.

[53] Lei Yang, Seo Jin Park, Mohammad Alizadeh, Sreeram Kannan, and David Tse. DispersedLedger: High-Throughput byzantine consensus on variable bandwidth networks. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 493–512, Renton, WA, April 2022. USENIX Association.

[54] Thomas Yurek, Licheng Luo, Jaiden Fairoze, Aniket Kate, and Andrew Miller. hbacss: How to robustly share many secrets. *Cryptology ePrint Archive*, 2021.

[55] Thomas Yurek, Zhuolun Xiang, Yu Xia, and Andrew Miller. Long live the honey badger: Robust asynchronous {DPSS} and its applications. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 5413–5430, 2023.

Appendix

1. Additional Preliminaries

1.1. Definitions of Agreement Primitives. We describe functionalities for the agreement primitives, following the descriptions from [18], [19].

Reliable Broadcast. We describe the functionality $\mathcal{F}_{\mathsf{rbc}}$ for reliable broadcast. When a party P_s inputs a value v to the functionality as the sender, we will say that "P_s (reliably) broadcasts value v". Moreover, when some party P_j receives an output v in a reliable broadcast functionality with sender P_i, we will say that "P_j receives output v from P_i's reliable broadcast", and we will omit specifying the sender if the context is clear. We refer the reader to [8], [32] for the details of $\mathcal{F}_{\mathsf{rbc}}$.

Multi-Value Validated Byzantine Agreement. We use the functionality $\mathcal{F}_{\mathsf{mvba}}$ introduced in [27].

Functionality $\mathcal{F}_{\mathsf{mvba}}$

$\mathcal{F}_{\mathsf{mvba}}$ proceeds as follows, running with a predicate function $\mathrm{Predicate}$, parties $P_1, \ldots, P_n$, and the ideal adversary $\mathcal{S}$. Let $\mathsf{I} = \mathsf{H}$, where H is the set of honest parties. Initialize x_i to $\perp$ for each party P_i and the output y to $\perp$. Let the message length be L, corruption threshold be t.

1: Upon receiving a message $m \in \{0,1\}^L$ from party P_i, do as follows.
- If any party or $\mathcal{S}$ has received an output y, then ignore this message; otherwise, set $x_i = m$.
- Send m to $\mathcal{S}$.

2: If there exists a subset $\mathcal{I} \subset [n]$ such that $|\mathcal{I}| \geq n - t$ and $\mathrm{Predicate}(x_i) = 1$ for all $P_i \in \mathcal{I}$, sample $j \xleftarrow{\$} \mathcal{I}$ and set $y = x_j$. Then functionality outputs y as a request-based delayed output to all parties.
- Upon receiving $(\texttt{Decide}, y')$ from $\mathcal{S}$, if the output has not been delivered to all parties and $\mathrm{Predicate}(y') = 1$, change the output of all parties by y'. Otherwise, ignore this message.

Other Functionalities. For the functionalities of $\mathcal{F}_{\mathsf{ba}}$, $\mathcal{F}_{\mathsf{acs}}$, $\mathcal{F}_{\mathsf{coin}}$, we refer the reader to [8], [37] for more details.

2. Construction of ACSS-Id

In this section, we recap how to construct ACSS-Id based on techniques in [8], [13], [49]. The functionality of $\mathcal{F}_{\mathsf{ACSS\text{-}id}}$ is as follows.

This construction is based on zero-knowledge proof (dZK). This dZK proof a prover to distribute a degree-d polynomial $f(\cdot)$, and each party can verify whether the shares received from the prover are correct. [13] provides a construction with amortized $\mathcal{O}(n \log^2 n)$ communication bits in the random oracle model. We also attach the functionality of dZK proof as follows.

Functionality $\mathcal{F}_{\mathsf{dZK}}$

Public Input: $(\alpha_0, \ldots, \alpha_n), d, N$
$\mathcal{F}_{\mathsf{dZK}}$ runs with parties $\mathsf{P} = \{P_1, \ldots, P_n\}$, a prover $P \in \mathsf{P}$, and an adversary $\mathcal{S}$.

1: Upon receiving polynomials $f_1(x), \ldots, f_N(x)$ from prover P, record them.

2: Upon receiving $(\texttt{VerifyDZK}, s_1, \ldots, s_N, \alpha_k)$ from a party, if $k \in [n]$, $f_1(x), \ldots, f_N(x)$ are of degree-d and $s_1, \ldots, s_N$ equal $f_1(\alpha_k), \ldots, f_N(\alpha_k)$, send a requested-based delayed output true to this party. Otherwise, send a request-based delayed output false to this party.

Based on $\mathcal{F}_{\mathsf{dZK}}$ and the ACSS-Id protocol in [49], we can construct $\Pi_{\mathsf{ACSS\text{-}id}}$ as follows, which realize $\mathcal{F}_{\mathsf{ACSS\text{-}id}}$. In the following, we will use Π to denote the ACSS-Id protocol in [49]. During Π, when a party terminates the sharing phase but considers his shares to be incorrect, he will consider the dealer to be corrupted. Later, when he complains to the dealer, all parties can trigger the "unhappy" path to identify the corrupted party. During the termination phase, all parties will invoke an instance of $\mathcal{F}_{\mathsf{ra}}$, which denotes a reliable agreement protocol. When all parties terminate $\mathcal{F}_{\mathsf{ra}}$ with an output, it is guaranteed that at least $t + 1$ honest parties provide this output as inputs for $\mathcal{F}_{\mathsf{ra}}$.

Protocol $\Pi_{\mathsf{ACSS\text{-}id}}$: ACSS with Identifiable Abort

Public Inputs: $(\alpha_0, \ldots, \alpha_n), t, N$.

Distribution Phase

1: **Distributing Shares.** D invoke the ACSS-Id protocol Π in [49] to distribute shares to all parties. All parties will not trigger the "unhappy" path during Π.

2: **Supporting Efficient PubRec.** D divides these sharings into $t + 1$ groups, each of size $L' = N/(t+1)$.

Denote the degree-t polynomials in the k-th group as $f_1^{(k)}(x), \ldots, f_{L'}^{(k)}(x)$, where $k \in [t+1]$. For each $\ell \in [L']$, we define a degree-(t,t) bivariate polynomial $g_\ell(x,y)$ as follows:

$$g_\ell(x,y) := f_\ell^{(1)}(x) + f_\ell^{(2)}(x) \cdot y + \cdots + f_\ell^{(t+1)}(x) \cdot y^t$$

For each $i \in [n]$, D invokes an instance of $\mathcal{F}_{\mathsf{dZK}}$ with input $g_1(x,\alpha_i), \ldots, g_{L'}(x,\alpha_i)$, denoted by $\mathcal{F}_{\mathsf{dZK}}^{(i)}$.

Verification Phase

3: For each party, when he terminates Π invoked by D, he checks his output received from Π:
- If he receives his shares from Π, he records his shares and proceeds.
- Otherwise, he sets his output as $(\texttt{Corrupt}, D)$.

4: If party P_j has recorded his shares in Step 1, he computes $\{g_\ell(\alpha_j,\alpha_i)\}_{\ell\in[L']}$ for each $i \in [n]$. Then P_j sends $(\texttt{VerifyDZK}, \{g_\ell(\alpha_j,\alpha_i)\}_{\ell\in[L']}, \alpha_j)$ to $\mathcal{F}_{\mathsf{dZK}}^{(i)}$ and records the output.

Termination Phase

5: All parties jointly invoke an instance of $\mathcal{F}_{\mathsf{ra}}$, each party who receives true from all $\{\mathcal{F}_{\mathsf{dZK}}^{(i)}\}_{i=1}^n$ sets his input as 1.

6: When all parties terminate $\mathcal{F}_{\mathsf{ra}}$ with 1, parties who get shares will output their shares. Each party that does not have shares will output $(\texttt{Corrupt}, D)$.

Upon receiving a request $(\texttt{Accusation}, P_i, D)$ from the environment, if all parties have terminated the sharing phase, they do the following.

Accusation Phase

1: All parties trigger the "unhappy" path for P_i and learn either P_i or D is corrupted.

Upon receiving a request $\texttt{Public-Recon}$ from the environment, if all parties have terminated the sharing phase, they do the following.

Efficient Public Reconstruction Phase

1: Each party P_i who has set his input as 1 for $\mathcal{F}_{\mathsf{ra}}$ will send $\{g_\ell(\alpha_i,\alpha_j)\}_{\ell\in[L']}$ to each party P_j.

2: When P_i receives $\{g_\ell(\alpha_j,\alpha_i)\}_{\ell\in[L']}$ from P_j, he sends $(\texttt{VerifyDZK}, \{g_\ell(\alpha_j,\alpha_i)\}_{\ell\in[L']}, \alpha_j)$ to $\mathcal{F}_{\mathsf{dZK}}^{(i)}$ and checks whether the output of $\mathcal{F}_{\mathsf{dZK}}^{(i)}$ is true. If true, he accepts $\{g_\ell(\alpha_j,\alpha_i)\}_{\ell\in[L']}$. Otherwise, he ignores these messages.

3: When P_i accepts $\{g_\ell(\alpha_j,\alpha_i)\}_{\ell\in[L']}$ from $t+1$ distinct P_j, he reconstructs $\{g_\ell(\alpha_0,\alpha_i)\}_{\ell\in[L']}$ and sends them to all parties.

4: When P_i receives $\{g_\ell(\alpha_0,\alpha_j)\}_{\ell\in[L']}$ from P_j, he records it. When P_i succeeds in reconstructing $\{g_\ell(\alpha_0,y)\}_{\ell\in[L']}$ by using online error correction, he outputs the coefficients of $\{g_\ell(\alpha_0,y)\}_{\ell\in[L']}$.

The communication complexity of $\Pi_{\mathsf{ACSS\text{-}id}}$ is $\mathcal{O}(Ln)$ field elements plus $\mathcal{O}(\kappa n^2 \log^2 n)$ bits. The additive overhead comes from n instances of $\mathcal{F}_{\mathsf{dZK}}$. The efficient public reconstruction will cost $\mathcal{O}(Ln+n^2)$ field elements. For the accusation phase, when all parties trigger the "unhappy" path for one party, the communication complexity is $\mathcal{O}(Ln)$ field elements plus $\mathcal{O}(\kappa n^2 \log n)$ bits. Recall that the communication complexity of the original ACSS protocol in [49] is $\mathcal{O}(Ln^2)$ field elements plus $\mathcal{O}(\kappa n^3 \log n)$ bits, this is because all parties will trigger "unhappy" path for $\mathcal{O}(n)$ distinct parties to help all of them recover their shares. While if we only want to accuse the dealer of being corrupted, one "unhappy" path is enough.

3. Security Analysis of DPSS

We prove lemma 1 and start with the construction of the ideal adversary $\mathcal{S}$ as follows.

Simulator $\mathcal{S}_{\mathsf{RandSh}}$

Simulation of Π_{RandSh}

1: For each instance of $\mathcal{F}_{\mathsf{ACSS\text{-}id}}$ invoked by dealer $P_i \in \mathsf{P}_1$:
- If P_i is honest, randomly sample shares of corrupted parties and send them to corrupted parties.
- Otherwise, honestly emulate $\mathcal{F}_{\mathsf{ACSS\text{-}id}}$ and record the output of each honest party.

2: For each dealer P_i, when he terminates his $\mathcal{F}_{\mathsf{ACSS\text{-}id}}$, $\mathcal{S}$ honestly emulates $\mathcal{F}_{\mathsf{coin}}$, randomly samples r as the random challenge and sends it to corrupted parties on behalf of $\mathcal{F}_{\mathsf{coin}}$.

3: For each dealer P_i, if P_i is corrupted, $\mathcal{S}$ honestly follows the protocol on behalf of each honest party. If not $f_\ell^{(i)}(\alpha_0) = g_\ell^{(i)}(\alpha_0)$ for all $\ell \in [0, L']$ but $\tilde{f}_*^{(i)}(\alpha_0) = \tilde{g}_*^{(i)}(\alpha_0)$, $\mathcal{S}$ aborts the simulation. If P_i is honest, $\mathcal{S}$ does the following.

(1). Compute $f_*^{(i)}(\alpha_j)$ for each corrupted $P_j \in \mathsf{P}_1$. Then randomly sample a degree-t polynomial $f_*^{(i)}(x)$ based on points of corrupted parties.

(2). For each honest party P_j, send $f_*^{(i)}(\alpha_j)$ to all corrupted parties in P_1 on behalf of P_j.

(3). When each honest party gets $f_*^{(i)}(\alpha_0)$, send $f_*^{(i)}(\alpha_0)$ to all corrupted parties on behalf of this honest party.

$\mathcal{S}$ does the same simulation for $g_*^{(i)}(\alpha_0)$.

4: $\mathcal{S}$ honestly follows the protocol, emulates $\mathcal{F}_{\mathsf{acs}}$ and learns a set D of size $2t_1+1$ of successful dealers.

5: $\mathcal{S}$ computes corrupted parties' shares of $[f_{\ell,k}(\alpha_0)]_{t_1}$ and $[g_{\ell,k}(\alpha_0)]_{t_2}$ for all $\ell \in [L'], k \in [t_1+1]$.

Simulator $\mathcal{S}_{\mathsf{PubRecPE}}$

Simulation of Π_{PubRecPE}

1: $\mathcal{S}$ takes the whole sharings $[s_1]_t, \ldots, [s_L]_t$ as inputs. For each honest party who does not have shares, $\mathcal{S}$ honestly follows the protocol.

2: $\mathcal{S}$ honestly follows $\Pi_{\mathsf{BatchPubRec}}$, takes each honest party's shares of $[s_1]_t, \ldots, [s_L]_t$ as input and gets the output of each honest party.

3: $\mathcal{S}$ honestly follows $\Pi_{\mathsf{Agreement}}$ (will honestly emulate $\mathcal{F}_{\mathsf{ba}}, \mathcal{F}_{\mathsf{mvba}}$) and gets the output.
- If the output is $s_1, \ldots, s_L$, $\mathcal{S}$ terminates.
- Otherwise, the output is the identity of a corrupted party. $\mathcal{S}$ honestly emulates $\mathcal{F}_{\mathsf{ACSS\text{-}id}}$ invoked by this dealer and sends his secrets to all corrupted parties on behalf of $\mathcal{F}_{\mathsf{ACSS\text{-}id}}$. All honest parties get shares distributed by this dealer and will not fail due to this dealer next time.

4: $\mathcal{S}$ does the following to update the whole sharings $[s_1]_t, \ldots, [s_L]_t$, denote the sharings distributed by this corrupted dealer P_i as $[x_1]_t, \ldots, [x_{L/(t+1)}]_t$, for each

honest party:

(1). Let D be the set of dealers, compute a vector e_ℓ such that $e_\ell[i] = x_\ell - [x_\ell]_t$ for all $\ell \in [L/(t+1)]$ and $e_\ell[j] = 0$ for the rest of $j \in \mathsf{D}$.

(2). For each $\ell \in [L/(t+1)]$, compute $\{E_{\ell,k}\}_{k=1}^{t_1+1} = \boldsymbol{V} \cdot \{[e_\ell[j]]_{t_1}\}_{j\in\mathsf{D}}$. Then transform $\{E_{\ell,k}\}_{k\in[t+1],\ell\in[L/(t+1)]}$ to $E_1, \ldots, E_L$.

(3). For each $\ell \in [L]$, set $[s_\ell]_t = [s_\ell]_t + E_\ell$.

Then $\mathcal{S}$ moves back to Step 1.

Simulator $\mathcal{S}$

Simulation of Π_{DPSS}

1: $\mathcal{S}$ sends $(\texttt{Transfer}, \mathsf{P}_2)$ to $\mathcal{F}_{\mathsf{DPSS}}$ on behalf of each honest party in P_1.
2: $\mathcal{S}$ invokes $\mathcal{S}_{\mathsf{RandSh}}$ and gets corrupted parties' shares of $[r_\ell]_{t_1}, [r_\ell]_{t_2}$ for all $\ell \in [L]$.
3: For each corrupted party in P_1, $\mathcal{S}$ computes his shares of $[b_\ell]_{t_1} = [s_\ell]_{t_1} + [r_\ell]_{t_1}$ for all $\ell \in [L]$. Then $\mathcal{S}$ randomly samples $\{[b_\ell]_{t_1}\}_{\ell=1}^{L}$, then generates the whole sharings $\{[b_\ell]_{t_1}\}_{\ell=1}^{L}$ based on secrets $\{[b_\ell]_{t_1}\}_{\ell=1}^{L}$ and shares of corrupted parties.
4: $\mathcal{S}$ invokes $\mathcal{S}_{\mathsf{PubRecPE}}$ and takes the whole sharings $\{[b_\ell]_{t_1}\}_{\ell=1}^{L}$ as inputs.
5: For each honest party $P_i \in \mathsf{P}_1$, $\mathcal{S}$ computes $B(\alpha_i)$ and sends it to all corrupted parties in P_2 on behalf of P_i. Then $\mathcal{S}$ delivers the output $\texttt{Term}$ from $\mathcal{F}_{\mathsf{DPSS}}$ to P_i.
6: When each honest party in P_2 gets $b_1, \ldots, b_L$, for each honest party P_i who can compute his shares of $[r_\ell]_{t_2}$, $\mathcal{S}$ delivers the output from $\mathcal{F}_{\mathsf{DPSS}}$ to P_i. Otherwise, $\mathcal{S}$ sends $(\texttt{success}, P_i)$ to $\mathcal{F}_{\mathsf{DPSS}}$.
7: $\mathcal{S}$ outputs the views of $\mathcal{A}$.

The hybrid arguments are as follows.

$\mathbf{Hyb}_0$: In this hybrid, we consider the execution in the real world.

$\mathbf{Hyb}_1$: In the following, we focus on the simulation Π_{RandSh}.

$\mathbf{Hyb}_{1.1}$: In this hybrid, for each honest dealer, $\mathcal{S}$ first randomly samples shares of corrupted parties, then generates the whole sharings based on shares of corrupted parties. Based on the property of Shamir sharings, this makes no difference. The distributions of $\mathbf{Hyb}_{1.1}$ and $\mathbf{Hyb}_0$ are identical.

$\mathbf{Hyb}_{1.2}$: In this hybrid, we change the generation of honest parties' shares of $[f_0^{(i)}]_{t_1}$ distributed by honest P_i. We first delay the generation of honest parties' shares until Step 5, since this share has not been used so far, this makes no difference. Then $\mathcal{S}$ first compute corrupted parties' shares on $f_*^{(i)}(x)$, then randomly samples a degree-t_1 polynomial as $f_*^{(i)}(x)$ based on shares of corrupted parties. Then $\mathcal{S}$ computes $f_0^{(i)}(x) = f_*^{(i)}(x) - \sum_{\ell=1}^{L'} r^\ell \cdot f_\ell^{(i)}(x)$. The difference between $\mathbf{Hyb}_{1.1}$ and $\mathbf{Hyb}_{1.2}$ is $\mathcal{S}$ first samples $f_0^{(i)}(x)$ or $f_*^{(i)}(x)$, since they are all randomly sampled, the distributions of $\mathbf{Hyb}_{1.2}$ and $\mathbf{Hyb}_{1.1}$ are identical.

$\mathbf{Hyb}_{1.3}$: In this hybrid, $\mathcal{S}$ do the same thing as $\mathbf{Hyb}_{1.2}$ to change the generation of honest parties' shares of $[g_0^{(i)}]_{t_2}$ distributed by honest P_i. The distributions of $\mathbf{Hyb}_{1.3}$ and $\mathbf{Hyb}_{1.2}$ are identical.

$\mathbf{Hyb}_{1.4}$: In this hybrid, we change the generation of the honest parties' shares of $\{[f_{\ell,k}(\alpha_0)]_{t_1}\}_{k=1}^{t_1+1}$. $\mathcal{S}$ first computes corrupted parties' shares of $\{[f_{\ell,k}(\alpha_0)]_{t_1}\}_{k=1}^{t_1+1}$, then randomly samples the whole $\{[f_{\ell,k}(\alpha_0)]_{t_1}\}_{k=1}^{t_1+1}$ based on shares of corrupted parties. Let H denote the set of the first t_1+1 honest parties in D and C denote the set of the rest of parties in D, Since:

$$\{[f_{\ell,k}(\alpha_0)]_{t_1}\}_{k=1}^{t_1+1} = \boldsymbol{V}^{\mathsf{H}} \cdot \{[f_\ell^{(j)}(\alpha_0)]_{t_1}\}_{j\in\mathsf{H}} + \boldsymbol{V}^{\mathsf{C}} \cdot \{[f_\ell^{(j)}(\alpha_0)]_{t_1}\}_{j\in\mathsf{C}}$$

Since $\boldsymbol{V}$ is a Vandermonde matrix and $\boldsymbol{V}^{\mathsf{H}}$ is a invertible matrix, $\mathcal{S}$ can computes $\{[f_\ell^{(j)}(\alpha_0)]_{t_1}\}_{j\in\mathsf{H}}$ based on $\{[f_{\ell,k}(\alpha_0)]_{t_1}\}_{k=1}^{t_1+1}$ and $\{[f_\ell^{(j)}(\alpha_0)]_{t_1}\}_{j\in\mathsf{C}}$. The difference between $\mathbf{Hyb}_{1.4}$ and $\mathbf{Hyb}_{1.3}$ is $\mathcal{S}$ first samples $\{[f_\ell^{(j)}(\alpha_0)]_{t_1}\}_{j\in\mathsf{H}}$ or $\{[f_{\ell,k}(\alpha_0)]_{t_1}\}_{k=1}^{t_1+1}$, which makes no difference since there exist a one to one map between them. The distributions of $\mathbf{Hyb}_{1.4}$ and $\mathbf{Hyb}_{1.3}$ are identical.

$\mathbf{Hyb}_{1.5}$: In this hybrid, $\mathcal{S}$ does the same thing as $\mathbf{Hyb}_{1.4}$ to change the generation of honest parties' shares of $\{[g_{\ell,k}(\alpha_0)]_{t_2}\}_{k=1}^{t_1+1}$. The distributions of $\mathbf{Hyb}_{1.5}$ and $\mathbf{Hyb}_{1.4}$ are identical.

$\mathbf{Hyb}_{1.6}$: $\mathcal{S}$ no longer generates honest parties' shares distributed by honest dealers since they are never used. The distributions of $\mathbf{Hyb}_{1.6}$ and $\mathbf{Hyb}_{1.5}$ are identical.

$\mathbf{Hyb}_{1.7}$: In this hybrid, $\mathcal{S}$ aborts if there exists one corrupted dealer P_i such that $f_\ell^{(i)}(\alpha_0) \neq g_\ell^{(i)}(\alpha_0)$ for some $\ell \in [0, L']$ but $\tilde{f}_*^{(i)}(\alpha_0) = \tilde{g}_*^{(i)}(\alpha_0)$. In this case, that means:

$$\sum_{\ell=0}^{L'} (f_\ell^{(i)}(\alpha_0) - g_\ell^{(i)}(\alpha_0)) \cdot r^\ell = 0$$

Then r can be considered as a root for equation $\sum_{\ell=0}^{L'} (f_\ell^{(i)}(\alpha_0) - g_\ell^{(i)}(\alpha_0)) \cdot x^\ell = 0$, since r is randomly sampled, the probability for this case is at most $L'/|\mathbb{F}|$. We take union bound for all corrupted dealers, then the probability is at most $L/|\mathbb{F}|$. Therefore, the distributions of $\mathbf{Hyb}_{1.7}$ and $\mathbf{Hyb}_{1.6}$ are statistically close, and the statistical error is bounded by $L/|\mathbb{F}|$.

$\mathbf{Hyb}_2$: In this hybrid, in Π_{DPSS}, we change the generation of honest parties' shares of $[r_\ell]_{t_1}$, $\mathcal{S}$ first computes corrupted parties' shares of $[s_\ell]_{t_1}$, then he randomly samples the whole $[b_\ell]_{t_1}$ based on shares of corrupted parties. With the whole sharing $[b_\ell]_{t_1}$, $\mathcal{S}$ computes each honest party's shares of $[r_\ell]_{t_1} = [b_\ell]_{t_1} - [s_\ell]_{t_1}$. The difference between $\mathbf{Hyb}_2$ and $\mathbf{Hyb}_{1.3}$ is $\mathcal{S}$ first samples honest parties' shares of $[b_\ell]_{t_1}$ or $[r_\ell]_{t_1}$, which makes no difference since they are both randomly sampled. The distributions of $\mathbf{Hyb}_2$ and $\mathbf{Hyb}_{1.7}$ are identical.

$\mathbf{Hyb}_3$: In this hybrid, in $\Pi_{\mathsf{PubRec\text{-}PE}}$, when each party needs to locally update his shares of $[s_\ell]_{t_1}$, he follows $\mathcal{S}_{\mathsf{PubRecPE}}$ to do update. The difference between $\mathbf{Hyb}_{3.1}$ and $\mathbf{Hyb}_2$ is whether $\mathcal{S}$ needs honest parties' shares, and $\mathcal{S}$ no longer needs them now. The distributions of $\mathbf{Hyb}_3$ and $\mathbf{Hyb}_2$ are identical.

$\mathbf{Hyb}_4$: In this hybrid, $\mathcal{S}$ no longer generates shares of honest parties since they are never used and $\mathcal{S}$ invokes $\mathcal{F}_{\mathsf{DPSS}}$ to deliver the output to each honest party. The difference between $\mathbf{Hyb}_4$ and $\mathbf{Hyb}_3$ is the whole random sharings among parties in P_2 are sampled by $\mathcal{S}$ or $\mathcal{F}_{\mathsf{DPSS}}$, which makes no difference. The distributions of $\mathbf{Hyb}_4$ and $\mathbf{Hyb}_3$ are identical.

Note that $\mathbf{Hyb}_4$ corresponds to the ideal world, then Π_{DPSS} securely computes $\mathcal{F}_{\mathsf{DPSS}}$.